GENERAL SIR JOHN KOTELWALA DEFENCE UNIVERSITY

FACULTY OF COMPUTING



BITCOIN PRICE FORECASTING USING DEEP LEARNING TIME-SERIES ANALYSIS

FINAL PROJECT REPORT

Group No. 08

Registration No	Name	Degree Program
D/BIS/24/0016	PYB Karunasena	BSc (Hons) Information Systems
D/BIT/24/0054	DMNWK Ranasinghe	BSc (Hons) Information Technology
D/BSE/25/0001	KDU Rashmika	BSc (Hons) Software Engineering
D/DBA/25/0003	DS Weerasooriya	BSc (Hons) Data Science and Business Analytics

Abstract

Short term Bitcoin prices are very volatile and noisy and exact price predictions for a few minutes are not reliable. This project developed CryptoForecast, a deep-learning system for predicting short-term Bitcoin behavior using historical BTC/USDT 15-min candles from Binance. Early experiments showed that accurate price prediction was not reliable so the goal was changed to predicting the direction of the price change and level of market activity for the next 15 to 90 minutes.

The final data set included 317,838 clean candles from August 2017 to September 2026. Missing candles were not filled and data was split into continuous segments. The window of each model input was 64 candles and forty-nine backward looking features were computed. We trained a multi-task Gated Recurrent Unit (GRU) network with a return head, a direction head (DOWN, NEUTRAL, UP) and a regime head (QUIET, ACTIVE) as an ensemble of three seeds. The data was split chronologically and purged between splits and 24 automated checks were in place to ensure no future information leaked into the features, targets or scaling.

During the test period (May 2025 – September 2026), the regime head achieved a mean accuracy of 0.648, compared to a baseline of 0.498. The direction head achieved a macro-F1 of 0.419 to 0.428 compared to 0.226 to 0.233 for the majority-class baseline, but was heavily biased toward NEUTRAL. The return head was slightly worse than forecast mean. The model was deployed through a React web dashboard and a terminal dashboard that track their own calls. The project shows that a recurrent network is able to find real signal about market activity but very little about exact returns. There is no claim of trading profit.

Keywords: Bitcoin; time series prediction; GRU; deep learning; volatility regime; data leakage

Table of Contents

Abstract i
Table of Contents ii
List of Figures iv
List of Tables v
List of Abbreviations vi
Chapter 1 Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Aim and Objectives 2
1.5 Scope 2
1.6 Significance 3
1.7 Overview of the Report 3
Chapter 2 Literature Review 4
2.1 Classical Time-Series Forecasting 4
2.2 Recurrent Networks, LSTM and GRU 4
2.3 Related Work on Bitcoin Forecasting 4
2.4 Evaluation Problems in Financial Machine Learning 5
2.5 Data Source 5
2.6 Comparison of Approaches and Project Gap 5
Chapter 3 Methodology and Requirements Analysis 6
3.1 Development Approach 6
3.2 Changes from the Original Plan 6
3.3 Functional Requirements 7
3.4 Non-Functional Requirements 7
3.5 Users 8
3.6 System Requirements 8
Chapter 4 System Analysis and Design 9
4.1 System Architecture 9
4.2 Data Design 10
4.3 Leakage-Safe Design 10
4.4 Target Design 11
4.5 Model Design 11
4.6 Prediction Store Design 12
4.7 API and Dashboard Design 13

4.8 Summary of Main Design Decisions 14
Chapter 5 Implementation 15
5.1 Development Environment and Technologies 15
5.2 Data Acquisition 15
5.3 Cleaning and Gap Handling 16
5.4 Feature Engineering 17
5.5 Target Engineering 18
5.6 Windows, Split and Scaling 19
5.7 Model and Training 20
5.8 Evaluation Module 21
5.9 Inference, API and Dashboards 21
5.10 Problems Found and Fixed During Development 23
Chapter 6 Testing and Evaluation 25
6.1 Testing Strategy 25
6.2 Leakage Audit 25
6.3 API, Store and Dashboard Tests 26
6.4 Test Cases 26
6.5 Model Evaluation Method 27
Chapter 7 Results and Discussion 28
7.1 Regime Head 28
7.2 Direction Head 29
7.3 Return Head 32
7.4 Validation and Test Compared 33
7.5 Effect of the Confidence Floor 33
7.6 Actual and Predicted Direction Over Time 35
7.7 Dashboard Output 35
7.8 Discussion 36
Chapter 8 Conclusion and Future Work 37
8.1 Summary 37
8.2 Achievement of Objectives 37
8.3 Main Contributions 38
8.4 Limitations 38
8.5 Future Work 38
References 40

List of Figures

Figure 4.1: End-to-end workflow of the implemented system

Figure 4.2: Chronological split of the usable windows

Figure 4.3: Architecture of one ensemble member

Figure 5.1: Data volume at each preparation stage

Figure 5.2: Direction class balance at the 15m horizon

Figure 5.3: Simple view of the React web dashboard

Figure 5.4: Advanced view of the React web dashboard

Figure 5.5: Terminal dashboard (live trend read)

Figure 7.1: Regime head accuracy on the test set

Figure 7.2: Direction head macro-F1 on the test set

Figure 7.3: Per-class recall of the direction head (test set)

Figure 7.4: Confusion matrix of the 15m direction head (test set)

Figure 7.5: Calibration of the direction head (test set)

Figure 7.6: Return head MAE against the baseline (test set)

Figure 7.7: Effect of raising the confidence floor (test set)

Figure 7.8: Actual and predicted 15m direction on the test split (generated by plot_trend.py)

List of Tables

Table 2.1: Comparison of forecasting approaches considered in this project

Table 3.1: Main changes between the proposal and the final implementation

Table 3.2: Functional requirements of the implemented system

Table 3.3: Non-functional requirements

Table 3.4: Software environment used for the reported results

Table 4.1: Main columns of the predictions table

Table 4.2: API endpoints

Table 4.3: Design decisions and reasons

Table 5.1: Technologies used and their role in the project

Table 5.2: Result of the continuity audit

Table 5.3: Feature groups

Table 5.4: Direction thresholds (training quantiles) and regime thresholds

Table 5.5: Chronological split

Table 5.6: Model and training settings

Table 5.7: Training runs of the three ensemble members (CPU only)

Table 6.1: Summary of the leakage audit

Table 6.2: Test cases and results taken from the saved logs

Table 7.1: Regime head accuracy on the test set

Table 7.2: Direction head results on the test set

Table 7.3: Per-class results of the direction head at 15 and 90 minutes (test set)

Table 7.4: Return head MAE (% return) on the test set

Table 7.5: Validation and test results for the main metrics

Table 8.1: Objective achievement

Table A.1: The 49 model features

Table B.1: Main files of the final code

List of Abbreviations

API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
ATR	Average True Range
BTC	Bitcoin
CPU	Central Processing Unit
EMA	Exponential Moving Average
GRU	Gated Recurrent Unit
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MACD	Moving Average Convergence Divergence
MAE	Mean Absolute Error
OHLCV	Open, High, Low, Close, Volume
REST	Representational State Transfer
RMSE	Root Mean Square Error
RSI	Relative Strength Index
SSE	Server-Sent Events
UTC	Coordinated Universal Time
USDT	Tether (US dollar stablecoin)

Chapter 1 Introduction

1.1 Background

Bitcoin is the best known cryptocurrency and is traded around the clock on many exchanges. Its price is affected by trading volume, market sentiment, news, regulation and speculation, so it can move sharply within a few minutes. Traditional linear forecasting methods such as moving averages and ARIMA assume fairly stable statistical behaviour, which Bitcoin prices do not show. For this reason, many recent studies have tested neural networks, especially recurrent networks, which can learn patterns directly from past data.

This project, named CryptoForecast by our group, studies short-term Bitcoin behaviour using a deep learning time-series model. The final system uses 15-minute BTC/USDT candles from Binance and a Gated Recurrent Unit (GRU) network that reads the previous 64 candles (16 hours) and produces predictions for the next 15 to 90 minutes.

1.2 Problem Statement

Short term bitcoin prices are noisy, volatile, and affected by factors not visible in historical prices. The problem in the original proposal was to predict the next closing price from recent OHLCV data. In the first experiments, the group found that it was not possible to predict an exact price for a very short period accurately, because of the noise in the data. The problem was thus reformulated in the progress report and the final implementation: Can we extract useful information about the short-term motion of Bitcoin, such as the direction and how active the market is likely to be, solely from the historical market data?

1.3 Motivation

Short-term movements are often analyzed by retail traders and students using charts and a few indicators. A model that learns from many indicators at the same time can help them understand the market better. At the same time, many forecasting projects report strong results that disappear once future information has leaked into the training data. The group wanted to build a system well tested and honestly reported, including parts that did not work well.

1.4 Aim and Objectives

Aim: To develop and evaluate a deep learning system for predicting short-term price direction and market activity from historical BTC/USDT 15-minute market data, and present the predictions through a dashboard that assists but does not replace human decision-making. The objectives were:

The specific objectives were:

1. To collect historical BTC/USDT 15-minute OHLCV data from the Binance public data service and prepare it for sequence modelling.
2. To design backward-looking technical and market features and build fixed-length input windows of 64 candles.
3. To build leakage-safe targets for six horizons (15 to 90 minutes) and a chronological train, validation and test split.
4. To train a GRU network that predicts future return, direction (DOWN, NEUTRAL, UP) and market regime (QUIET, ACTIVE).
5. To evaluate the model against simple baselines using suitable metrics, and to report weaknesses clearly.
6. To build a dashboard that shows the market, the model output and the model's own running record of past calls.
7. To make clear that the system is a decision-support tool and not an automatic trading system.

1.5 Scope

The project includes one asset (BTC/USDT), one interval (15 minutes) and one exchange (Binance spot). Only historical price, volume, number of trades and time features are used in the model. It does not rely on news, sentiment on social media or on macroeconomic data. It is not an order entry system, does not size positions and does not contain a profit back test with fees and slippage. These limits are discussed again in Chapter 8.

1.6 Significance

The project provides the group with a comprehensive example of a time-series machine learning workflow: from data to a working dashboard. It is primarily not a trading strategy, but rather a well tested pipeline that has 24 automated leakage checks written and passes, a test period read once, and every result compared to a simple baseline. The results indicate where a recurrent network detects real signal (market activity) and where it does not (exact return and direction).

1.7 Overview of the Report

A literature review of related work is provided in Chapter 2. The development approach and the requirements are explained in Chapter 3. Chapter 4 describes the design of the data pipeline, model and dashboard. Implementation is explained in Chapter 5. The testing and evaluation are described in Chapter 6. The results are presented and discussed in Chapter 7. The report ends with Chapter 8 which presents the limitations and recommendations for further studies.

Chapter 2 Literature Review

This Chapter provides a short introduction to the concepts behind the project. It is based on the references mentioned in the project proposal and selected from a few popular sources on Recurrent Networks and Financial machine learning.

2.1 Classical Time-Series Forecasting

ARIMA and moving-average models describe a series using its own past values and past errors. They work best when the series is close to stationary and the relationship is linear. Cryptocurrency prices change their level and volatility over time, so these assumptions are weak. The project proposal gave this as the main reason for moving to neural networks.

2.2 Recurrent Networks, LSTM and GRU

The recurrent neural network reads in the sequence a step at a time and keeps a hidden state. The vanishing gradient problem exists in ordinary recurrent networks which are difficult to train on long sequences. A Long Short-Term Memory (LSTM) unit was added to solve this issue [1] by adding gates. Later, the Gated Recurrent Unit (GRU) was introduced with reduced number of gates and parameters [2], and an empirical evaluation was performed that showed that gated units are more effective on sequence tasks than plain recurrent units [3]. The group decided to use the GRU model for this project because it is more easily trained than the LSTM on a CPU, and it was trained without a GPU.

2.3 Related Work on Bitcoin Forecasting

There are a number of studies on deep learning in Bitcoin. Liu et al. researched deep learning-based Bitcoin price prediction [4] and Adegboruwa et al. used an LSTM network to predict the bitcoin time-series [5]. We were inspired from these works to use recurrent networks in our proposal. Our project is different in two respects: First, it requires a much shorter time horizon (15 to 90 minutes) and second, it considers the design of the evaluation, in particular the prevention of data leakage, as a core component of the project.

2.4 Evaluation Problems in Financial Machine Learning

Financial returns are noisy, have heavy tails, and high volatility tends to be followed by high volatility, known as volatility clustering [6]. The above two facts affected our design. The project also predicts the direction of the next period, as the direction is difficult to predict, it also considers if the next period will be more active or less active than the recent past. Second is that labels constructed from future prices have a time overlap, and a random train / test split allows information to leak from one label to the other. To avoid this, López de Prado suggests purging and embargoing around split boundaries [7]. The project is based on this concept.

2.5 Data Source

Binance also releases historical kline (candlestick) data via bulk archives and via Binance's REST API [8]. In the project proposal, there was a reference to CoinMarketCap [9] but the development will be limited to Binance.

2.6 Comparison of Approaches and Project Gap

Table 2.1: Comparison of forecasting approaches considered in this project

Approach	Strength	Limitation for this problem
ARIMA / moving average	Simple and fast, easy to explain	Assumes linear, stable behaviour
Feedforward network	Simple baseline; no sequence memory	Treats the window as a flat vector; not implemented in the final project
LSTM	Gated memory for long sequences	More parameters and slower on a CPU
GRU (used)	Gated memory with fewer parameters	Still limited by the noise in the data

The gap we are addressing here is pragmatic. Many small projects estimate one price and report one error figure. This project predicts multiple horizons rather than a single horizon, separates the question of direction from the question of market activity, checks every result in relation to a baseline, and tests the pipeline for leakage.

Chapter 3 Methodology and Requirements Analysis

3.1 Development Approach

The project was implemented in three iterative stages. In the proposal stage the group aimed to predict the next closing price using feedforward, LSTM and GRU models. In the progress stage, the initial experiments with 15-minute data showed that the exact price prediction was unreliable, so the goal was changed to predict the price movement over several future candles and GRU was chosen as the main model. In the last phase, the data pipeline was reengineered with a focus on leakage prevention, the feature set was expanded to 49 features, and a market-regime output and a web dashboard were implemented.

An additional small logging script (runlog.py) was used to record important commands, which saves the exact terminal output into the logs folder of the repository. The results in this report are taken from these logs and from the saved metrics file, not from re-typed numbers.

3.2 Changes from the Original Plan

Table 3.1: Main changes between the proposal and the final implementation

Area	Original plan	Final implementation
Prediction target	Next closing price	Future % return, DOWN/NEUTRAL/UP direction and QUIET/ACTIVE regime for 15 to 90 minutes
Data	Historical OHLCV from an exchange API	317,838 clean Binance BTCUSDT 15m candles (Aug 2017 to Sep 2026)
Missing candles	Not discussed	Never filled; every gap starts a new segment
Models	Feedforward, LSTM and GRU compared	GRU ensemble of three seeds, compared with naive baselines
Dashboard	Streamlit	React web dashboard and a terminal dashboard, served by FastAPI
Metrics	RMSE, MAE, MAPE	Macro-F1, per-class recall, calibration, regime accuracy; MAE and RMSE for the return head

The progress report had filled 70 missing candles by interpolation in a 200,000-candle dataset. In the final version this was removed, because interpolated candles are not real market data and could mislead a sequence model. The final dataset is larger and covers the full Binance 15-minute history.

3.3 Functional Requirements

Table 3.2: Functional requirements of the implemented system

ID	Requirement
FR1	The system shall download BTCUSDT 15-minute candles from Binance without an API key.
FR2	The system shall validate the data, report missing candles and split the data into continuous segments.
FR3	The system shall compute 49 backward-looking features for every candle.
FR4	The system shall create future return, direction and regime targets for the required horizons.
FR5	The system shall build 64-candle windows that never cross a data gap and split them chronologically.
FR6	The system shall train GRU models with a return head, a direction head and a regime head.
FR7	The system shall evaluate models on validation and test data and compare them with baselines.
FR8	The system shall produce a six-horizon read from the newest candles using the frozen scaler and model.
FR9	The system shall record every call it makes and later mark it right or wrong when the outcome is known.
FR10	The system shall show the market, the prediction and the recorded track in a web dashboard and a terminal dashboard.

3.4 Non-Functional Requirements

Table 3.3: Non-functional requirements

Requirement	Description
Leakage safety	No feature, target, scaler or threshold may use information from after the time it is meant to predict.
Reproducibility	All settings are kept in one configuration file; random seeds are fixed; results can be regenerated by the provided scripts.
Transparency	Every result is shown next to a baseline, and weak results are labelled in the dashboard.
Hardware	Training and inference must run on a CPU without a GPU.
Robustness	Dashboard panels must show a safe empty state instead of failing when data is missing.
Maintainability	Data code uses only numpy, pandas and scikit-learn; deep learning libraries are imported only where needed.

3.5 Users

The target users are the same as the proposal, retail crypto traders that want a short term view, and students/researchers that want to study forecasting. The system only has one user type, a viewer. There is no login and no roles so a use case diagram would add little and is not included.

3.6 System Requirements

Table 3.4: Software environment used for the reported results

Item	**Version / detail**
Operating system	Windows 11
Python	3.13.12
Deep learning	TensorFlow 2.21.0 with Keras 3.14.0, CPU only
Data libraries	numpy, pandas, scikit-learn, requests
Dashboard (Python)	rich (terminal), FastAPI and uvicorn (API)
Dashboard (web)	React 18, Vite 5, lightweight-charts 4.2; Node 18 or later
Storage	SQLite (Python standard library) for the prediction store

Chapter 4 System Analysis and Design

4.1 System Architecture

This system has two parts. The first is an offline pipeline which prepares the data and trains the model. The second is a live part that downloads the newest candles, applies the same cleaning, features and scaler that was used in training, runs the model and shows the result in a dashboard. The workflow is shown in figure 4.1.

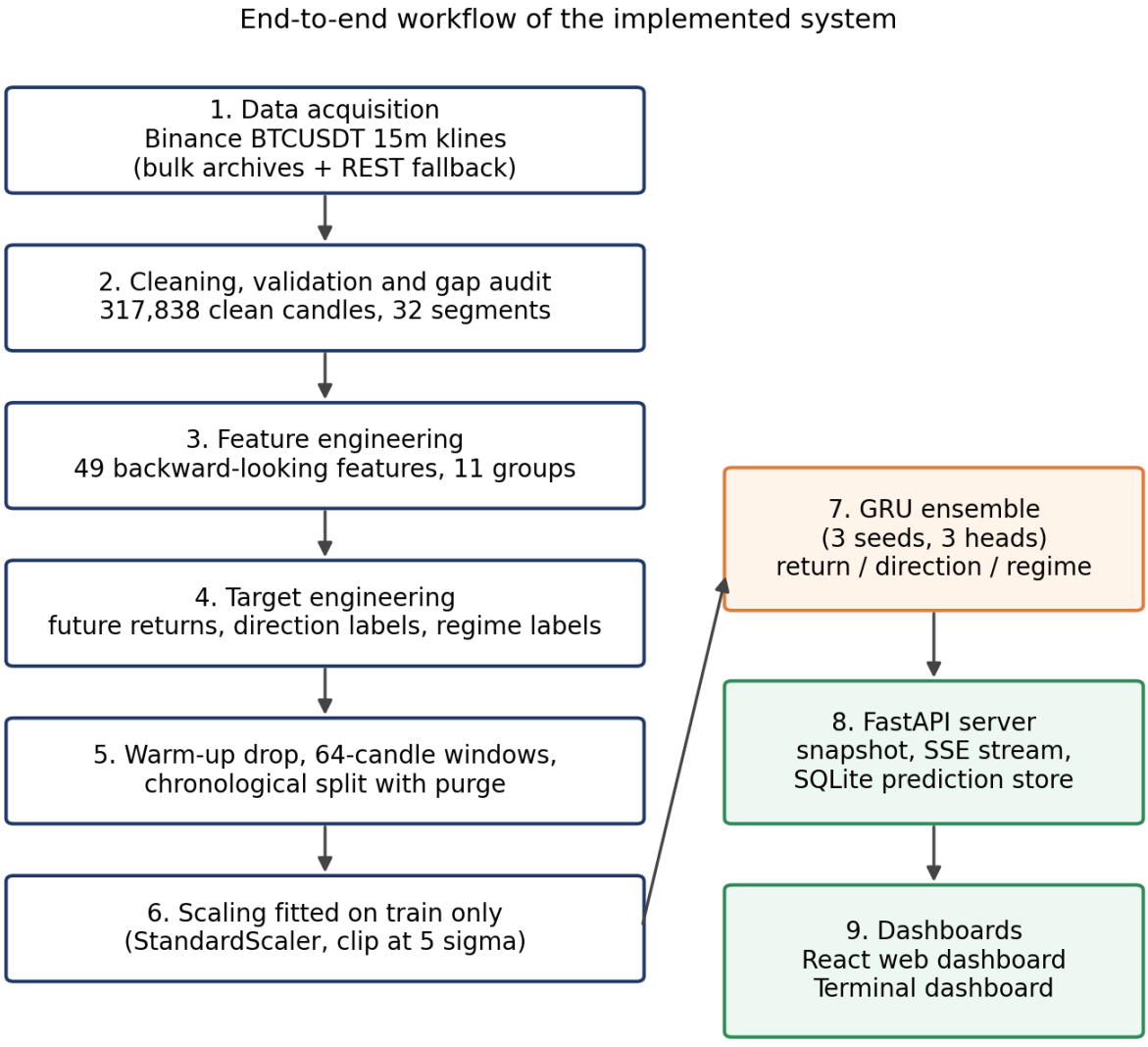


Figure 4.1: End-to-end workflow of the implemented system

Steps 1 to 6 belong to the data pipeline (pipeline.py). Steps 7 to 9 belong to the training, API and dashboard code (train.py, server.py, dashboard.py and the web folder). A single configuration file (config.py) holds every setting, so the pipeline, the notebooks and the dashboard all use the same values.

4.2 Data Design

Binance klines contain the open time, OHLC prices, volume, quote-asset volume, number of trades and taker-buy volumes. The project keeps ten of these columns (timestamp, open, high, low, close, volume, quote-asset volume, number of trades, taker-buy base volume and taker-buy quote volume). These data are in three stages: raw, processed and final. The last stage contains the scaled arrays, window start offsets, targets, timestamps, and a metadata file that contains all reported statistics.

Samples are not stored as one row per sample, but instead as a sequence of rows grouped into blocks and with a list of starting positions for each block. This is required as some of the windows covering a data gap are eliminated, creating gaps in the list of starts. A window is read as the rows starting with the start position and continuing thru the start position + 64.

4.3 Leakage-Safe Design

The group treated data leakage as the main design risk. Six mechanisms were built into the pipeline, and each one is verified by the audit script (Chapter 6):

1. Every feature at time t uses only rows up to t. No negative shifts, centred windows, back-filling or interpolation are used.
2. Future data is touched only when the targets are built, and a target that would span a data gap is set to missing.
3. Missing candles are never created. Every gap starts a new segment, and neither a window nor a target may cross a segment edge.
4. The split is chronological (70 % train, 15 % validation, 15 % test) and is never shuffled.
5. A purge of 70 samples (64-candle window plus 6-candle maximum horizon) separates the splits, so no training label is built from candles inside a validation or test window.
6. The scaler and the direction thresholds are fitted on the training split only and then frozen.

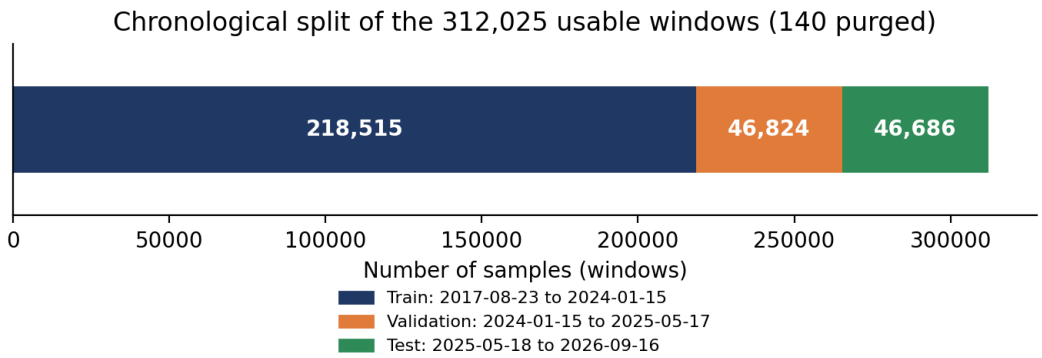


Figure 4.2: Chronological split of the usable windows

4.4 Target Design

Future return. For each horizon h from 1 to 6 candles, the target is 100 × (close[t+h] / close[t] − 1), which gives the percentage return 15, 30, 45, 60, 75 and 90 minutes ahead.

Direction. Each return is converted into DOWN, NEUTRAL or UP. The thresholds are quantiles of the training returns chosen so that NEUTRAL covers the middle 40 % of the training data (about 30 % DOWN, 40 % NEUTRAL, 30 % UP). These thresholds are then applied unchanged to validation and test data. The test period had smaller moves on average than the training period, so the same thresholds give about 24 % DOWN, 54 % NEUTRAL and 22 % UP at 15 minutes (Figure 5.2).

Market regime. The regime target answers a different question: will the average absolute 15-minute move over the next h candles be larger than the average of the previous 24 hours (96 candles)? The ratio of the two values is compared with its training median, giving two classes, QUIET and ACTIVE. The classes are balanced by construction, so 0.50 is a fair baseline. This target was added because the sign of a very short move is close to unpredictable, while volatility tends to cluster.

4.5 Model Design

The model is a multi-task GRU network. It has one shared recurrent trunk with two GRU layers, followed by three output heads. Sharing the trunk lets the direction and regime tasks shape the same internal representation.

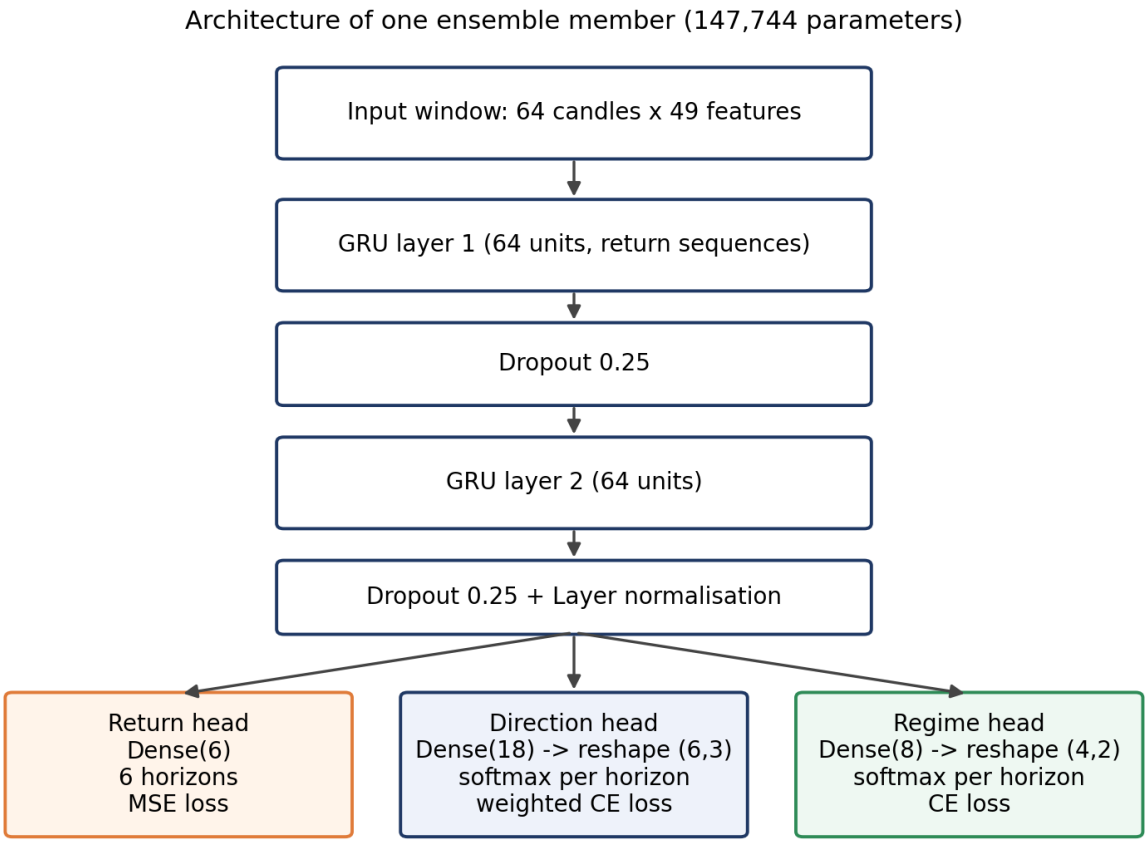


Figure 4.3: Architecture of one ensemble member

The final system is an ensemble of three such models trained with seeds 42, 43 and 44. Their softmax outputs are averaged. The group averaged the seeds instead of picking the best one, because the difference between seeds was within normal training noise, and choosing one would mean choosing noise.

4.6 Prediction Store Design

Every projection made by the dashboard is saved in a SQLite database. The table has one row per horizon, per head, per window, and the combination of anchor time, horizon and kind is unique. A repeated refresh therefore cannot create duplicate rows, and the real closing price is written into the same row once the target candle has formed.

Table 4.1: Main columns of the predictions table

Column	Meaning
anchor_ts, horizon, kind	Candle projected from, horizon label (15m to 90m) and head ('direction' or 'regime'); together they are unique
made_at, model	Time of the call and the model that made it
candles, target_ts	How many candles ahead, and the time of the target candle
predicted, confidence	Class chosen by the head and its probability
expected_pct, projected_close	Return head output and the close price it implies
activity	QUIET or ACTIVE from the regime head, otherwise empty
actual_close	Filled in later when the target candle has happened

4.7 API and Dashboard Design

The FastAPI server contains no prediction or scoring logic of its own. It drives the same Dashboard object used by the terminal dashboard and returns the result as JSON. The browser only draws what it receives. This keeps one single implementation of every rule, so the web and terminal views cannot disagree.

Table 4.2: API endpoints

Endpoint	Purpose
GET /api/snapshot	Everything the web dashboard needs in one request
GET /api/candles	Recent OHLCV candles
GET /api/prediction	Six-horizon read and the projected curve
GET /api/track-record	Resolved calls and running tallies
GET /api/metrics	Held-out test metrics from the saved evaluation file
GET /api/stream	Snapshot sent again as server-sent events
GET /api/health	Liveness, model name and last fetch time

The web dashboard has two views. The simple view, which opens by default, shows two plain-language answers, a chart and a six-row table. The advanced view adds the full chart layers, probability bars for both heads, two scorecards, the return distribution, the held-out metrics and the call log. The simple view states directly on the card that the direction question is close to a coin flip, so that simplifying the layout does not simplify the claim.

4.8 Summary of Main Design Decisions

Table 4.3: Design decisions and reasons

Decision	Reason
No raw price level as a feature	Bitcoin moved from about 4,000 to over 100,000 USDT; raw prices would let the network learn the price era instead of market behaviour
No interpolation of missing candles	Interpolated candles are not real market data
Regime head added	Direction has little signal at 15 minutes; volatility clustering gives a more predictable question
Class-weighted direction loss	Plain cross-entropy made the model say NEUTRAL almost every time
Early stopping on a combined score	The return head had no skill and dominated validation loss
Averaged three-seed ensemble	Differences between seeds were within noise

Chapter 5 Implementation

5.1 Development Environment and Technologies

The table below explains what each main technology is, why it was selected and how it was used.

Table 5.1: Technologies used and their role in the project

Technology	Why it was selected	How it was used
Python 3.13	Main language for data science and deep learning	Data pipeline, training, evaluation, API and terminal dashboard
pandas, numpy	Fast table and array handling	Cleaning, features, targets and windowing
scikit-learn	Standard scaling tools	StandardScaler fitted on the training split
TensorFlow / Keras	Mature library with GRU layers and CPU support	Building and training the multi-task GRU
Binance Vision and REST API	Public data, no API key	Bulk monthly archives, with REST for the newest candles
FastAPI and uvicorn	Light Python web framework with streaming support	Serving the snapshot and the server-sent event stream
SQLite	Built into Python, no installation	Storing every projection and its outcome
rich	Terminal tables and panels	Live terminal dashboard
React, Vite, lightweight-charts	Component based UI and a fast financial chart	Web dashboard with candlesticks and projection lines
Jupyter / Colab notebooks	Step by step inspection	Five notebooks for download, features, targets and training

5.2 Data Acquisition

The module data_download.py downloads Binance's monthly archive files, which needs one request per month instead of thousands of paged requests. It uses daily archives for the current month and the REST klines endpoint for the newest candles, with retries and back-off. It also handles a change that Binance made in 2025: timestamps in archives from January 2025 are in microseconds instead of milliseconds, and the loader converts both to UTC. The raw download contained 317,919 candles from 17 August 2017 to 16 September 2026.

5.3 Cleaning and Gap Handling

The cleaning step checks column types, converts numbers, removes invalid OHLC rows, sorts by time and removes duplicates. 81 rows with unreadable timestamps were dropped, leaving 317,838 clean candles. The gap audit then compared them with the 318,481 candles expected on a complete 15-minute grid.

Table 5.2: Result of the continuity audit

Item	**Value**
Candles present	317,838
Candles expected on the grid	318,481
Missing candles	643 (0.2019 %)
Discontinuities (gaps)	31, of which 30 were large (3 or more candles)
Largest gap	215 candles (8 to 10 February 2018)
Contiguous segments	32

In the progress report, 70 missing candles in a smaller dataset were filled by interpolation. In the final code this was replaced by the segment policy: a new segment starts at every gap, and no window or target crosses it. After rows were dropped for other reasons, segment numbers were recalculated, so the 32 segments became 56 continuous runs, and holes created by dropped rows also became boundaries.

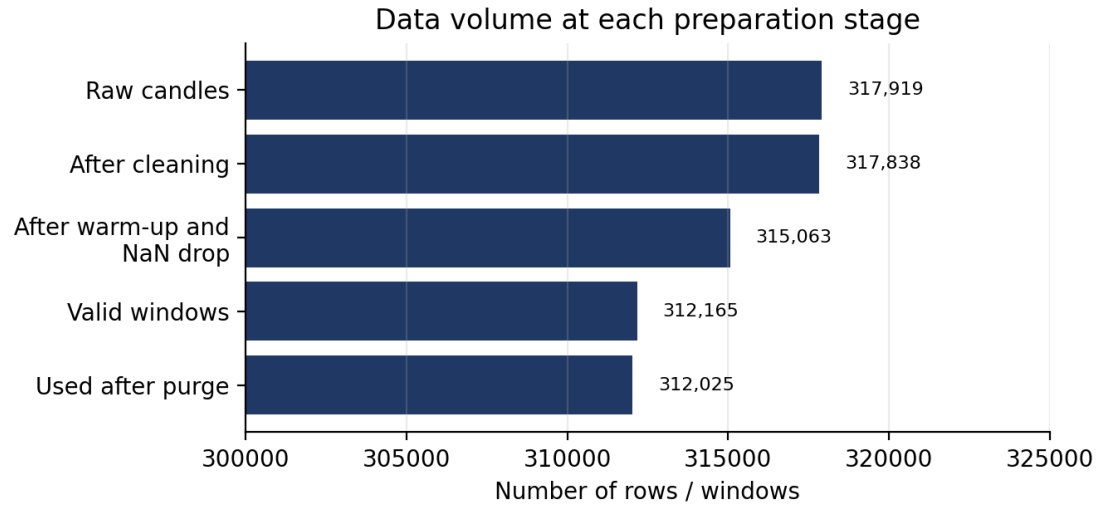


Figure 5.1: Data volume at each preparation stage

Figure 5.1 shows how the data shrinks through the pipeline. The first 500 rows were removed as warm-up so that indicators such as the 100-period EMA had converged. Rows with missing feature values (4,911 missing values across 7 columns) and the last 6 rows with no future close were removed, and 636 further targets were invalidated because they would have crossed a gap.

5.4 Feature Engineering

The module features.py creates 49 features in 11 groups. All of them are returns, ratios, distances, z-scores or volatilities. No absolute price level is used, and the raw OHLCV columns are kept only for auditing.

Table 5.3: Feature groups

Group	Count	Examples
Returns	6	Past returns over 1, 2, 4, 8, 16 and 32 candles
Volatility	6	Rolling volatility over 16, 32 and 96 candles, a volatility ratio, ATR (14) and ATR as a percentage
Trend	6	Distance of the close from EMA 20, 50 and 100; EMA ratios; EMA 20 slope
Momentum	4	RSI (14), MACD line, MACD signal and MACD histogram, normalised by price
Bollinger	4	Band width, position inside the bands and distances to the upper and lower bands
Volume	5	Volume change, ratios to the 16 and 96 candle averages, z-score, regime ratio
Trades	4	Trade-count change and ratios, trade-count z-score, average trade size ratio

Order flow	3	Taker-buy ratio, its deviation and its z-score
Candle shape	4	Candle range, body ratio, upper wick and lower wick
Range position	3	Position inside the 64-candle range, distance from the 24-hour high and low
Time	4	Sine and cosine of the hour of day and of the day of week (UTC)

The proposal and progress report planned seven features (EMA 20, EMA 50, RSI, MACD, MACD signal, ATR and percentage change). These indicators are all still present, but in normalised form.

5.5 Target Engineering

The thresholds below were calculated on the training split only. The 15-minute threshold, for example, is about −0.103 % and +0.109 %, so a candle return inside this band is NEUTRAL.

Table 5.4: Direction thresholds (training quantiles) and regime thresholds

Horizon	**DOWN below**	**UP above**	**Regime: ACTIVE if ratio above**
15m	−0.1033 %	+0.1087 %	0.7390
30m	−0.1389 %	+0.1491 %	0.8090
45m	−0.1634 %	+0.1796 %	not used
60m	−0.1841 %	+0.2034 %	0.8528
75m	−0.2045 %	+0.2288 %	not used
90m	−0.2220 %	+0.2505 %	0.8724

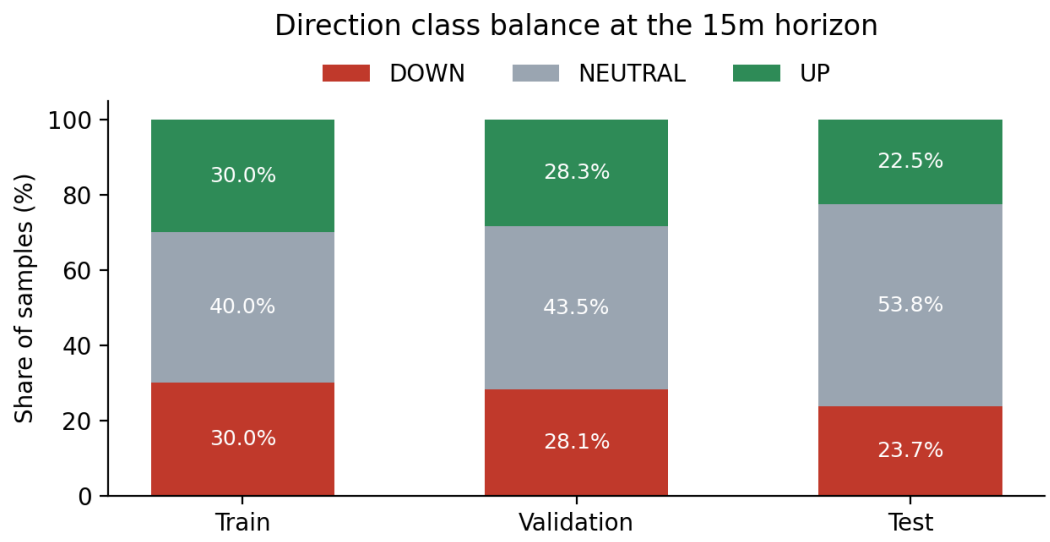


Figure 5.2: Direction class balance at the 15m horizon

Figure 5.2 shows that the thresholds give exactly 30/40/30 on the training split, but the test period contains many more NEUTRAL samples (53.8 %). This is a change in market behaviour between periods and it affects how the results should be read.

5.6 Windows, Split and Scaling

Each sample is a window of 64 consecutive candles with 49 features. Of 315,000 candidate windows, 312,165 were valid (no gap and a target available). After removing 70 samples at each of the two split boundaries, 312,025 were used.

Table 5.5: Chronological split

Split	Samples	Start	End
Train	218,515	2017-08-23	2024-01-15
Validation	46,824	2024-01-15	2025-05-17
Test	46,686	2025-05-18	2026-09-16
Purged	140	70 at each boundary	

A StandardScaler was fitted only on the rows used by training windows and then applied to all splits. Scaled values were clipped at ±5 standard deviations, which affected 0.329 % of the training values. Returns have heavy tails (the 15-minute return kurtosis is about 84), so clipping was used to keep training numerically stable. The return targets were also scaled for the regression head.

5.7 Model and Training

Table 5.6: Model and training settings

Setting	Value
Input	64 candles × 49 features
Recurrent layers	2 GRU layers, 64 units each
Regularisation	Dropout 0.25 after each GRU layer, layer normalisation
Outputs	Return (6), direction (6 × 3), regime (4 × 2)
Parameters	147,744 per ensemble member
Optimiser	Adam, learning rate 0.001
Batch size / max epochs	1024 / 25
Losses	MSE (return), class-weighted cross-entropy (direction), cross-entropy (regime)
Direction class weights	DOWN 1.047, NEUTRAL 0.907, UP 1.047 (strength 0.5)
Model selection score	0.5 × direction macro-F1 + 0.5 × regime accuracy on validation
Callbacks	Early stopping (patience 5), learning-rate halving (patience 2), best-model checkpoint

The most important design lines are the output heads. The direction output is reshaped to six groups of three before the softmax is applied, so that each horizon has its own probabilities:

```
d = Dense(n_h * 3, name="direction_logits")(x)
d = Reshape((n_h, 3), name="direction_reshape")(d)
direction_head = Activation("softmax", name="direction_head")(d)
```

Table 5.7: Training runs of the three ensemble members (CPU only)

Seed	Epochs run	Best epoch	Time (minutes)	Seconds/epoch
42	10	5	14.2	about 85
43	11	6	15.5	about 84
44	16	11	21.3	about 80

The three runs took about 51 minutes in total (the repository README reports 53 minutes wall clock including evaluation). Training used the Adam optimiser [10] in TensorFlow [11]. The validation selection score of the GRU was 0.5505

5.8 Evaluation Module

The script evaluate.py loads the saved models and the frozen artefacts and measures everything again from scratch. It does not change the model or choose any setting, because choosing a setting from test results would turn the test split into a second validation split. It reports regression metrics (MAE, RMSE, bias, correlation), direction metrics (accuracy, macro-F1, per-class precision and recall), confusion matrices, calibration, and a confidence-floor sweep. The script plot_trend.py draws the actual and predicted direction on a common time axis.

5.9 Inference, API and Dashboards

The terminal dashboard (dashboard.py) fetches the newest candles, passes them through the same cleaning, feature and scaling path as training, and shows the six-horizon read. Every new 15-minute candle produces a new set of calls. Each call is saved with the time at which it can be checked, and when that candle appears, the dashboard compares the result with the frozen thresholds and marks the call right or wrong. It shows this running hit rate next to the baseline.

The React dashboard (web folder) draws candlesticks, the projected path of the return head, a zero-change baseline, the recorded history of earlier projections, and green or red markers for calls that were right or wrong. A background thread in the server records projections every 30 seconds even when no browser is open, so closing a tab does not leave holes in the record. A gap longer than four candles is drawn as a break, which means only that the server was not running.

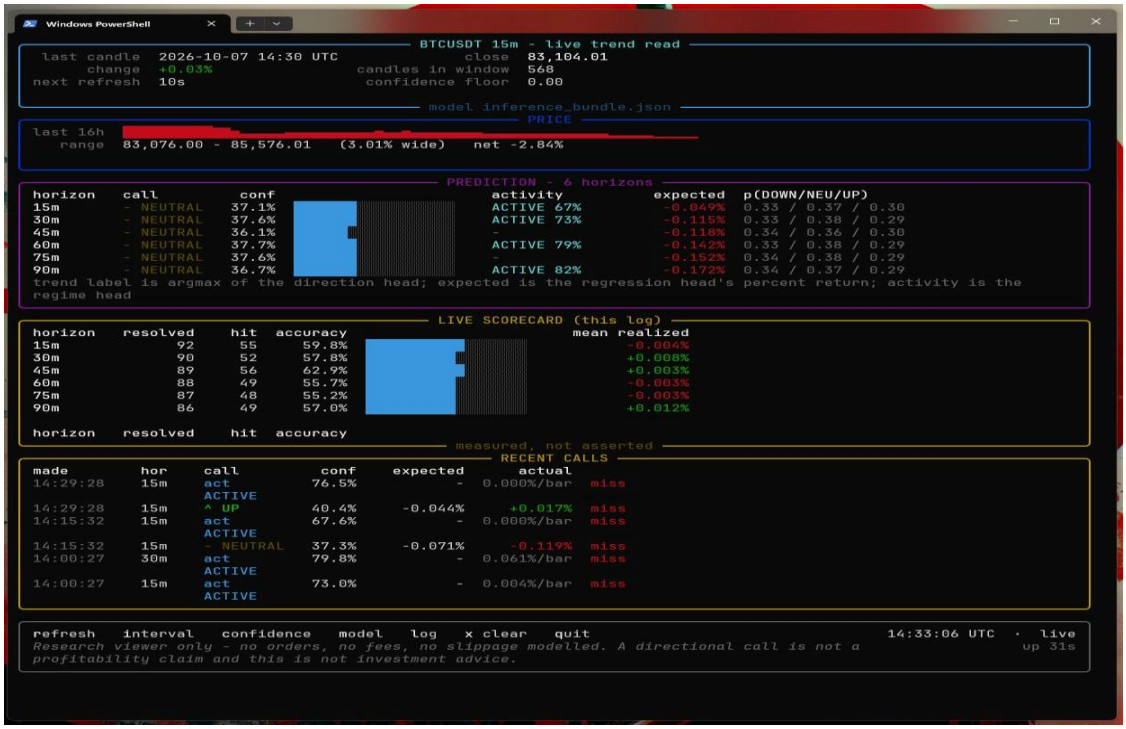


Figure 5.3: Simple terminal dashboard view

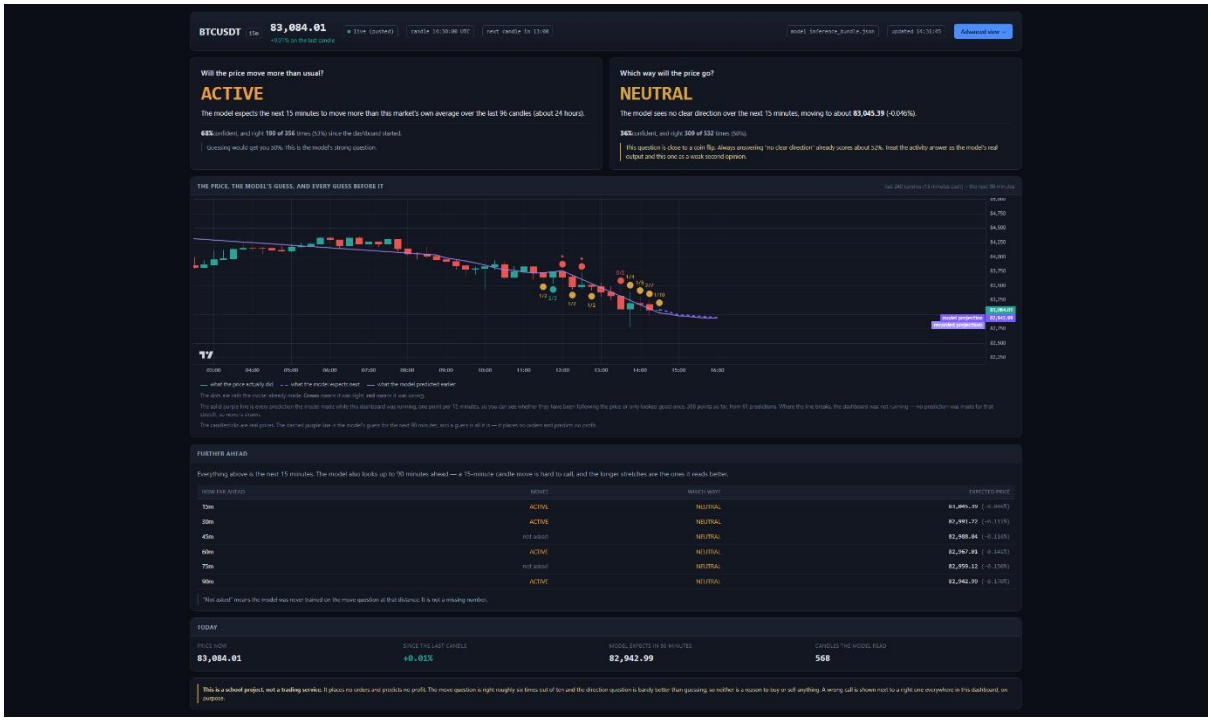


Figure 5.4: View of Live trend Dash Board

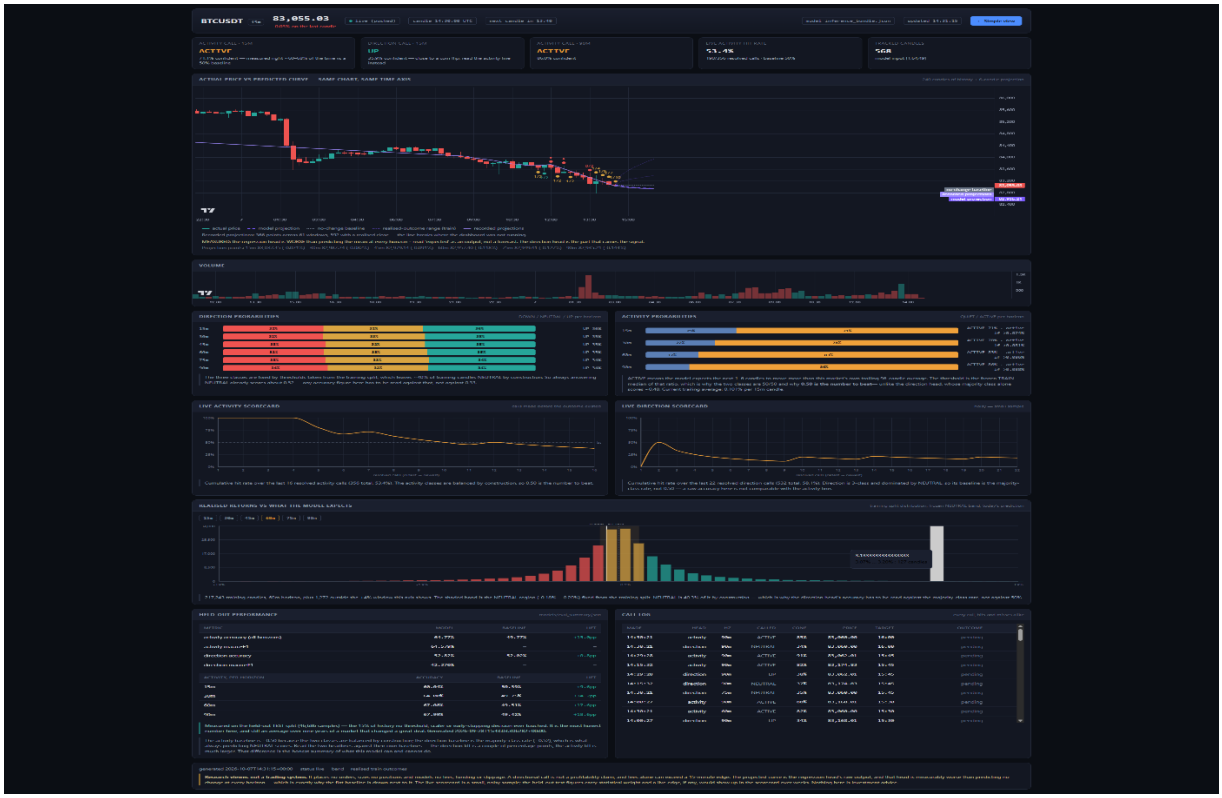


Figure 5.5: Terminal dashboard (live trend read)

5.10 Problems Found and Fixed During Development

- **Softmax shared across horizons.** An earlier version applied softmax to the flat 18-value vector and reshaped afterwards. Each horizon's probabilities then summed to about 1/6 instead of 1. The predicted class was still correct, which hid the problem, and it was found only because the displayed probabilities did not add up. The order was changed to reshape first, and the model builder now checks the sums.
- **NEUTRAL collapse.** With plain cross-entropy, the direction head almost always predicted NEUTRAL (test recall about 0.19 for DOWN and 0.21 for UP). Class weights were added. They improved recall but did not solve the problem.
- **Early stopping on the wrong signal.** The original validation loss was dominated by the return head, which has no skill. Early stopping was changed to the combined direction and regime score.
- **Dashboard marker crash.** The chart rejected markers that were not in time order because the server listed calls in the order they were issued. The marker builder now

sorts, merges calls on the same candle and ignores invalid times, and a separate check covers this case.

Chapter 6 Testing and Evaluation

6.1 Testing Strategy

Testing was done at three levels. First, the data pipeline was checked with an automated leakage audit that re-runs the real feature, target, scaler and threshold code on deliberately corrupted data. Second, the API, the prediction store and the dashboard components were checked with scripts that fail if any check fails. Third, the trained model was evaluated on held-out validation and test data and compared with simple baselines. The outputs of these runs were saved as logs in the repository (captured on 28 September 2026).

6.2 Leakage Audit

The script audit_leakage.py contains 24 checks, and all 24 passed. One example is the first check: every row from position 2,000 onwards was corrupted, and the features before that point were compared with the original values. The maximum difference was exactly 0, which shows that no feature looks forward in time.

Table 6.1: Summary of the leakage audit

Area (check numbers)	What was verified
Features (1, 16)	Features are backward-looking; none of the 21 target columns is among the 49 features
Targets (2, 3, 13, 21)	Future returns match the formula; targets across a gap are invalid; direction labels reproduce from the frozen thresholds; regime labels are gap-safe
Scaler and thresholds (4, 5, 12, 18)	Scaler and thresholds do not change when validation and test rows are corrupted; both come from the training split only
Split (6, 10, 11)	Train, validation and test follow each other in time; the 70-sample purge separates them (71 rows or 1,065 minutes at each boundary)
Windows (8, 9, 19, 20)	Array shapes match; no NaN or infinite values; offsets stay inside their blocks; no window crosses a gap (0 of 312,025 windows)
Artefacts and structure (7, 11a, 12a, 14, 15, 17, 22)	All files exist; stored arrays equal the scaled CSV values (largest difference 3.81e-06); every class is present; metadata is complete; the pipeline imports no deep learning framework; regime classes are balanced across splits (largest deviation 0.69 percentage points)

6.3 API, Store and Dashboard Tests

The API smoke test starts the server and calls every endpoint. It checks the model name, the number of horizons, that every horizon has a direction and a regression output, that the regime output exists only where it should, that candle times ascend and sit on 15-minute boundaries, and that the stream sends valid frames. The prediction store test uses a temporary database and checks that re-recording a window adds nothing, that the first write wins, that the real close is filled in once, that a gap in recording breaks the chart line, and that the read-only mode cannot write. The dashboard checks render every panel in Node against a captured real payload and against degenerate payloads (empty record, missing metrics, error frame), and confirm that the simple view opens by default.

6.4 Test Cases

Table 6.2: Test cases and results taken from the saved logs

ID	Feature	Test case	Expected result	Actual result	Status
T01	Leakage	Corrupt all rows after row 2,000 and recompute features	Earlier features unchanged	Maximum difference 0	Pass
T02	Leakage	Corrupt validation and test rows and refit scaler	Scaler statistics unchanged	Identical mean and scale	Pass
T03	Windows	Check every window for gap crossing	0 crossings	0 of 312,025	Pass
T04	Split	Check time order and purge	Train < validation < test, purge above 69 rows	71 rows at both boundaries	Pass
T05	Audit	Run all 24 checks	All pass	24 / 24 passed	Pass
T06	API	Call snapshot, candles, prediction, track-record, metrics	HTTP 200 and valid content	All returned 200	Pass
T07	API	Call candles with limit 5	Rejected (minimum is 10)	HTTP 422	Pass
T08	API	Open the event stream	Valid snapshot frames	2 frames parsed	Pass
T09	Store	Record the same window twice	No duplicate rows	Nothing added	Pass
T10	Store	Open in read-only mode and try to write	Write refused	30 rows before and after	Pass

T11	Dashboard	Render all panels with real and empty data	No exception	All components rendered	Pass
T12	Dashboard	Open the app	Simple view by default, toggle offered	As expected	Pass
T13	Markers	Build chart markers from out-of-order calls	Sorted and merged, no crash	10 calls to 6 markers	Pass

Only the tests above could be confirmed from the project files. If the group ran manual or user tests, they should be added to Table 6.2. These scripts are checks written for this project and are not a separate unit-testing framework.

6.5 Model Evaluation Method

The model was selected on validation data. The test split was read once by evaluate.py after training had finished. Each head was compared with a baseline that it has to beat:

- **Regime head:** accuracy against the majority-class rate, which is about 0.50 because the classes are balanced by construction.
- **Direction head:** macro-F1 against the majority-class macro-F1. Macro-F1 is used instead of accuracy because the NEUTRAL class is large, so plain accuracy looks better than the model really is.
- **Return head:** mean absolute error (MAE) against a model that always predicts the mean return.

Calibration was also measured, which shows whether a stated confidence of 70 % is really right about 70 % of the time.

Chapter 7 Results and Discussion

All numbers in this chapter are from the held-out test set (46,686 samples, 18 May 2025 to 16 September 2026) unless stated otherwise. They come from the saved evaluation log and metrics file.

7.1 Regime Head

Table 7.1: Regime head accuracy on the test set

Horizon	Majority baseline	GRU ensemble	Improvement over baseline
15m	0.5039	0.6004	+0.0964
30m	0.4975	0.6400	+0.1425
60m	0.4951	0.6708	+0.1757
90m	0.4942	0.6799	+0.1856
Mean	0.4977	0.6477	+0.1501

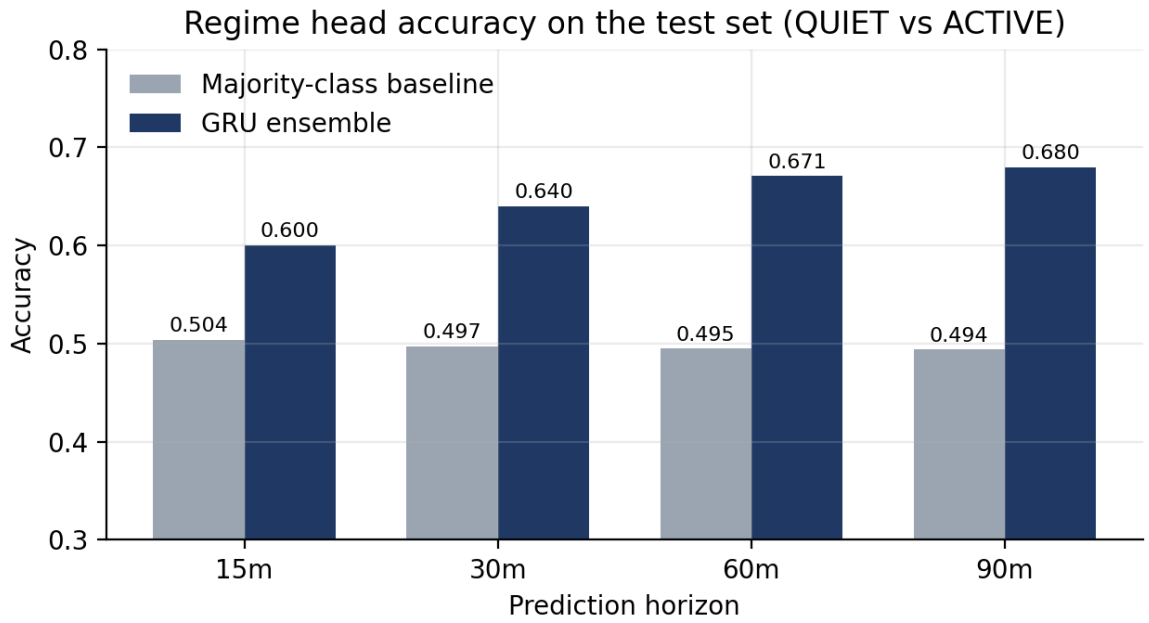


Figure 7.1: Regime head accuracy on the test set

The regime head was the strongest part of the model. It beat the baseline at every horizon, and the improvement became larger as the horizon became longer. At 90 minutes, QUIET precision and recall were 0.71 and 0.60, and ACTIVE precision and recall were 0.66 and 0.76, so both

classes were predicted reasonably and the model did not simply favour one class. The model was more likely to call ACTIVE correctly than QUIET at every horzon.

This result is expected because volatility tends to cluster [6]. It is important to be clear about what it means. The statement that the next window will be busier than the last 24 hours says nothing about whether the price will go up or down, so it is not a trading signal.

7.2 Direction Head

Table 7.2: Direction head results on the test set

Horizon	Accuracy	Majority accuracy	Macro-F1	Majority macro-F1	Ratio
15m	0.5450	0.5383	0.4279	0.2333	1.83
30m	0.5356	0.5221	0.4276	0.2287	1.87
45m	0.5299	0.5190	0.4250	0.2278	1.87
60m	0.5217	0.5125	0.4194	0.2259	1.86
75m	0.5204	0.5148	0.4211	0.2266	1.86
90m	0.5163	0.5144	0.4212	0.2264	1.86

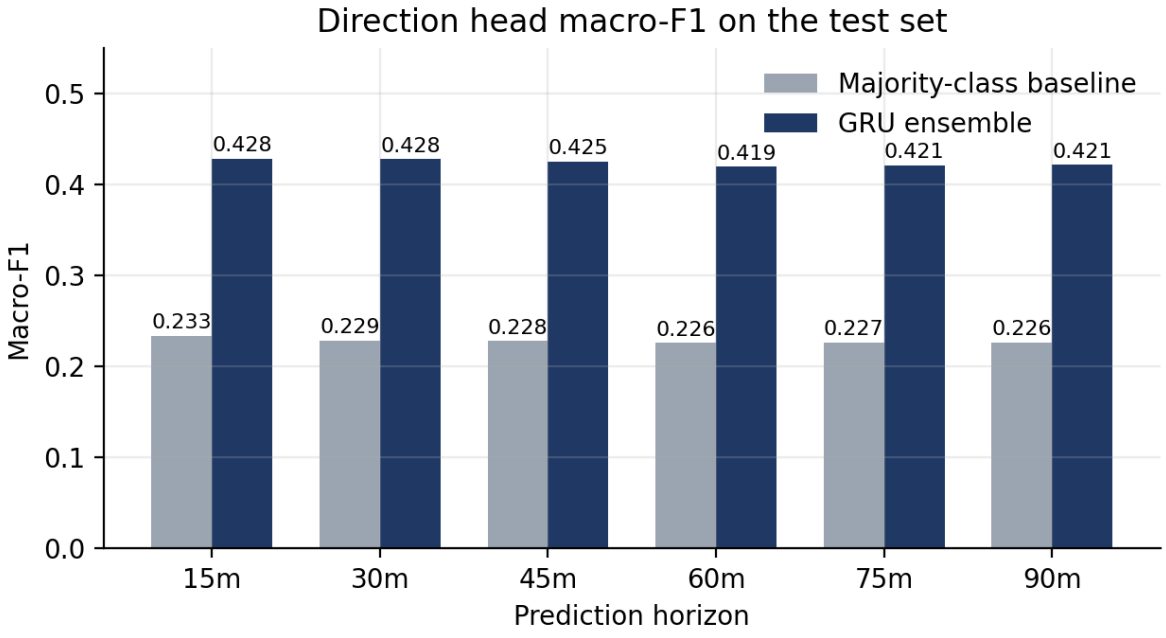


Figure 7.2: Direction head macro-F1 on the test set

The direction head beat the majority-class baseline on macro-F1 at all six horizons, but accuracy was only slightly above the always-NEUTRAL baseline (for example 0.5450 against 0.5383 at 15 minutes). The reason is shown by the per-class recall.

Table 7.3: Per-class results of the direction head at 15 and 90 minutes (test set)

Horizon	Class	Precision	Recall	F1
15m	DOWN	0.3586	0.2422	0.2892
15m	NEUTRAL	0.6322	0.8052	0.7083
15m	UP	0.3513	0.2414	0.2861
90m	DOWN	0.3464	0.2934	0.3177
90m	NEUTRAL	0.6089	0.7613	0.6766
90m	UP	0.3542	0.2171	0.2692

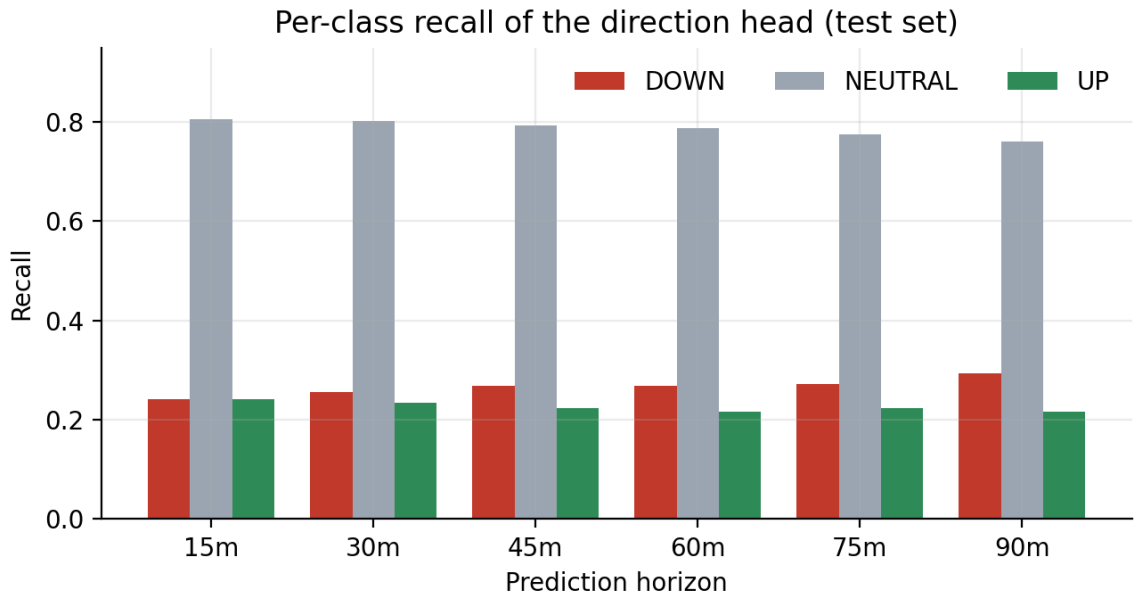


Figure 7.3: Per-class recall of the direction head (test set)

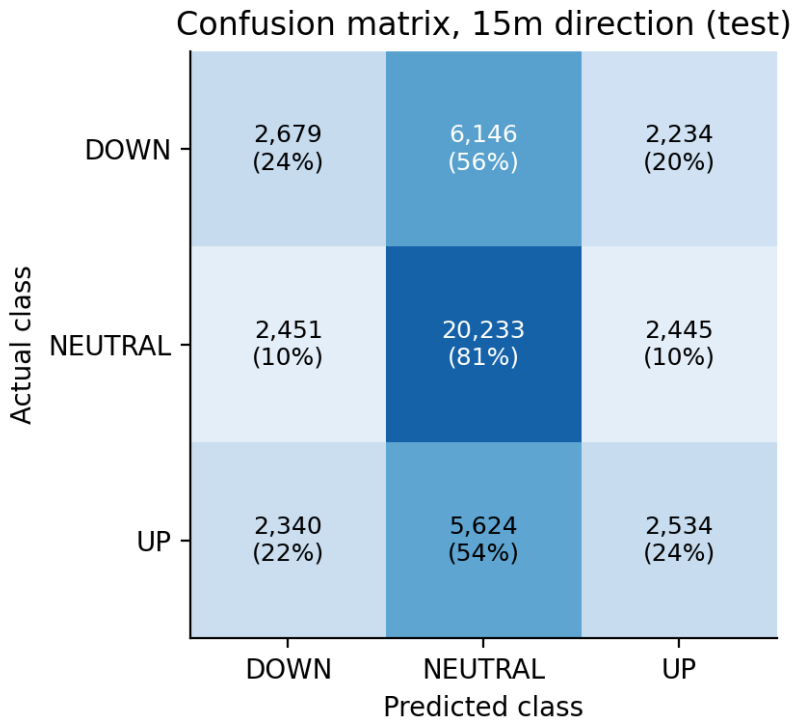


Figure 7.4: Confusion matrix of the 15m direction head (test set)

The model is strongly biased towards NEUTRAL. At 15 minutes it found about one in four real DOWN moves and one in four real UP moves, and it labelled more than half of them as NEUTRAL (Figure 7.4). The class weights reduced this bias compared with the earlier version, but the bias remains the main weakness of the direction head.

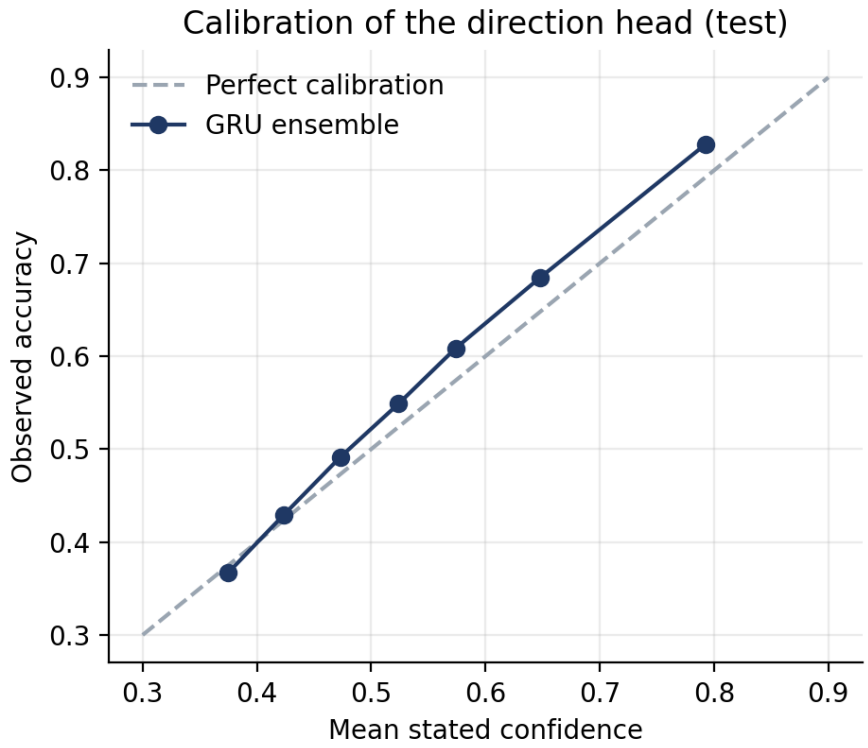


Figure 7.5: Calibration of the direction head (test set)

Confidence was informative. Accuracy rose from 0.37 in the lowest confidence bucket to 0.83 in the highest. The observed accuracy was slightly above the stated confidence in most buckets (by up to about 0.04), so the model was not over-confident on this test set.

7.3 Return Head

Table 7.4: Return head MAE (% return) on the test set

Horizon	Predict-the-mean	GRU return head	Correlation with actual
15m	0.145945	0.146489	0.0122
30m	0.205102	0.207241	0.0119
45m	0.249220	0.252593	0.0072
60m	0.287304	0.292507	0.0038
75m	0.321510	0.327982	0.0013
90m	0.352292	0.361520	−0.0046

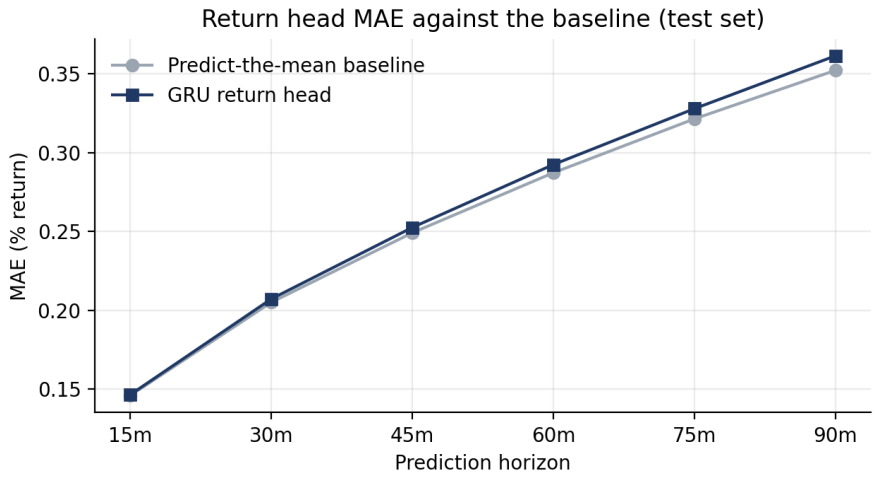


Figure 7.6: Return head MAE against the baseline (test set)

The return head was slightly worse than predicting the mean at every horizon, and its correlation with the true return was close to zero. This means the model cannot forecast the size of the return. The head was kept only as an auxiliary task to shape the shared trunk, and both dashboards label its output with this warning. This result agrees with the decision, taken after the progress report, to stop treating exact price prediction as the goal.

7.4 Validation and Test Compared

Table 7.5: Validation and test results for the main metrics

Metric	Validation	Test	Difference
Regime accuracy, mean of 4 horizons	0.6711	0.6477	−0.0234
Regime accuracy at 90m	0.7091	0.6799	−0.0292
Direction macro-F1 at 15m	0.4351	0.4279	−0.0072
Direction macro-F1 at 90m	0.4264	0.4212	−0.0052

Results on the test set were close to, but a little lower than, those on the validation set. This is the normal pattern when a model is selected on validation data, and it also shows that the test period was a different market period.

7.5 Effect of the Confidence Floor

The dashboard allows a confidence floor, below which a call is shown as NEUTRAL. This looks careful, so the group measured its effect.

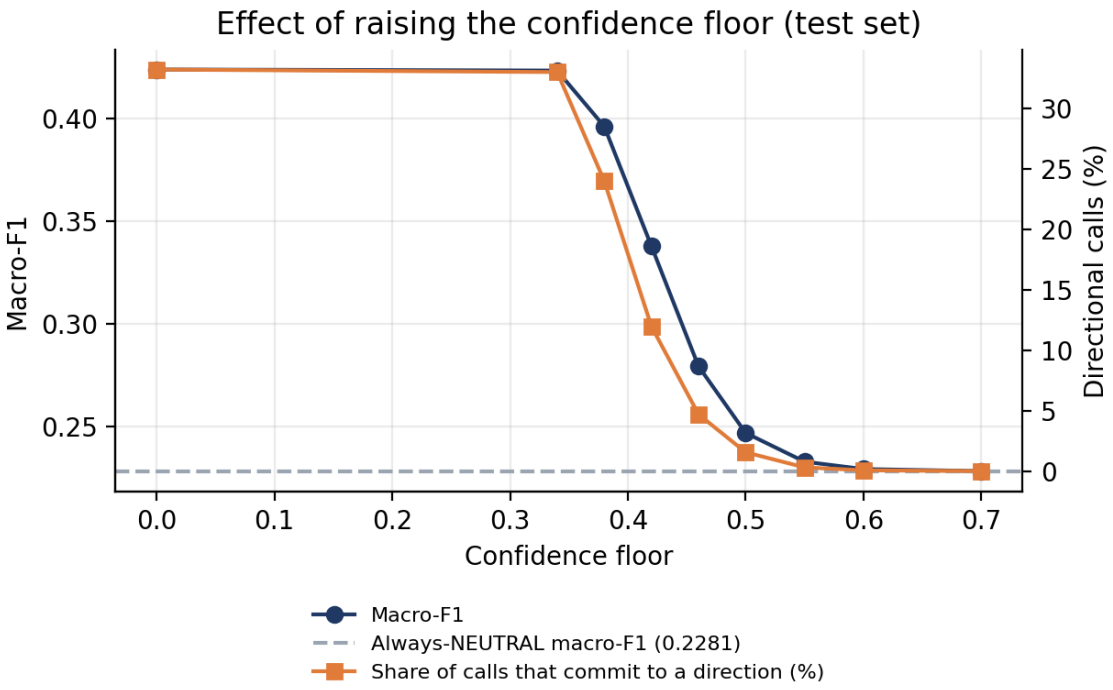


Figure 7.7: Effect of raising the confidence floor (test set)

Macro-F1 fell steadily as the floor increased, from 0.4237 with no floor to 0.2283 at a floor of 0.70, which is almost the always-NEUTRAL level of 0.2281. At the same time, accuracy hardly changed. A higher floor removes exactly the scarce DOWN and UP calls, so it hurts the weakest part of the model. The floor control was therefore kept as an option that can be tested, and the dashboard prints this caveat when a floor is used. For the regime head the opposite holds: choosing only high-confidence calls raised accuracy on the validation set from 0.67 to 0.81 at a floor of 0.70, while answering about 32 % of the time.

7.6 Actual and Predicted Direction Over Time

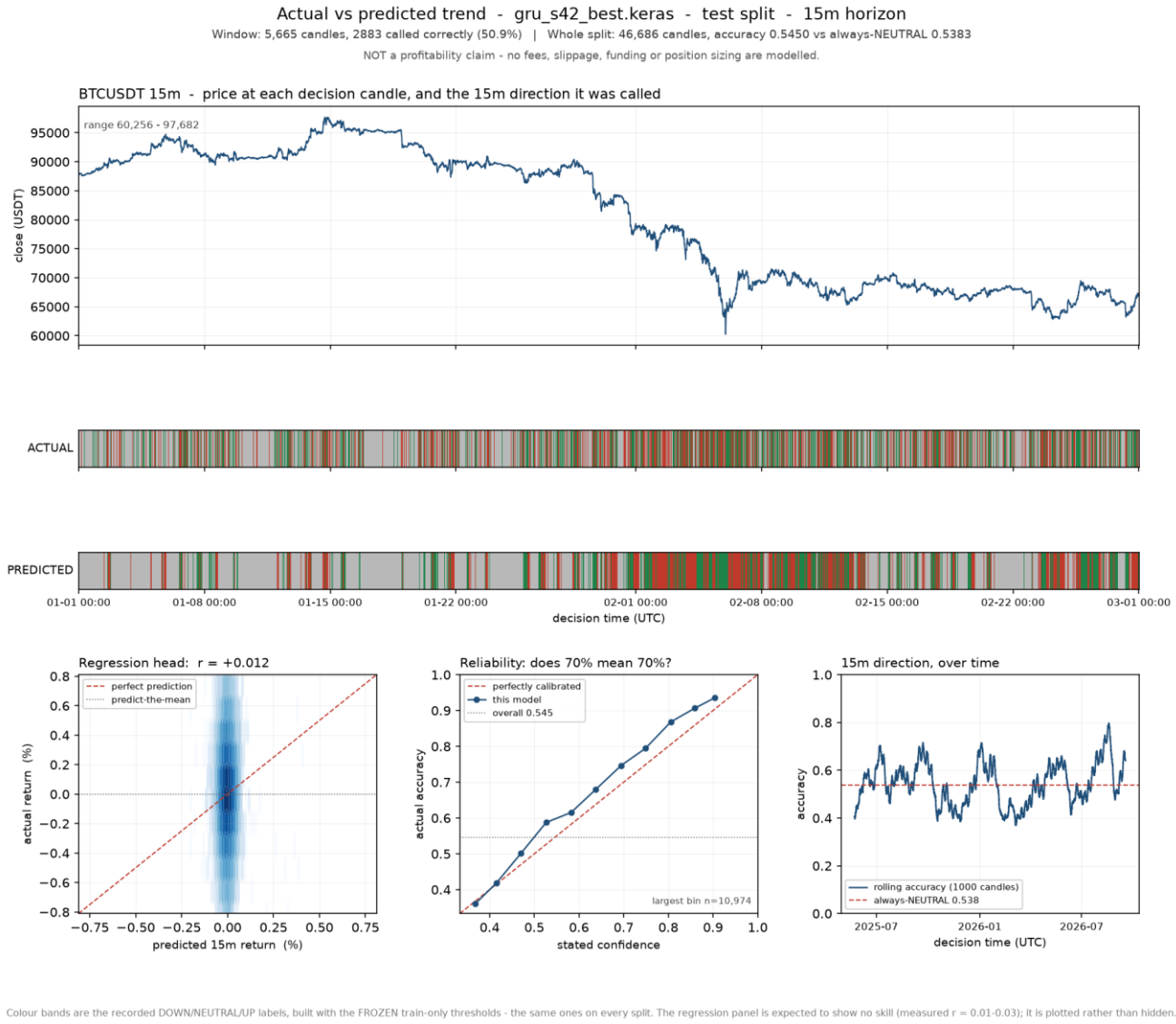


Figure 7.8: Actual and predicted 15m direction on the test split (generated by plot_trend.py)

Figure 7.8 shows the actual and predicted direction as colour bands under the price. The predicted band is mostly grey (NEUTRAL), while the actual band contains many more red and green stripes. This gap is the NEUTRAL bias seen in the confusion matrix. The rolling accuracy in the lower right panel moves widely, from about 0.4 to 0.8, around the always-NEUTRAL line, so the skill is small and not constant over time. The regression panel confirms that predicted returns are concentrated near zero.

7.7 Dashboard Output

The dashboards display the six-horizon read, the confidence of each call, the activity (QUIET or ACTIVE) at four horizons, the price chart with the projected path, and the live scorecard.

The scorecard compares the live hit rate with the baseline. In the saved logs it had only started to collect results (10 calls issued, none yet resolved), so no live accuracy figure can be reported.

7.8 Discussion

Strengths. The data pipeline is careful and well tested. The model finds real signal about market activity, with a mean improvement of 0.15 over the baseline on unseen data from a later period. The direction head also beats its baseline at all horizons. The system reports its own weaknesses instead of hiding them.

Weaknesses. The return head has no skill, and the direction head misses about three out of four real directional moves. A macro-F1 of about 0.42 and a regime accuracy of about 65 % are far from a trading edge. There is no backtest, so nothing in this report shows that the model would make a profit after fees and slippage.

Change in the test period. The same thresholds gave 40 % NEUTRAL on training data and 54 % on test data. The market in 2025 and 2026 appeared to move less in 15-minute steps than in earlier years, so the model was tested under conditions different from those in training. This is a realistic test but it also makes the results harder to compare with the validation set.

Chapter 8 Conclusion and Future Work

8.1 Summary

The project started as an attempt to predict the next Bitcoin closing price and developed into a leakage-audited system for short-term trend and activity prediction. The final system downloads about nine years of Binance 15-minute data, builds 49 features, trains a three-seed GRU ensemble with three outputs, and shows the result in a web dashboard and a terminal dashboard that keep a record of their own calls.

8.2 Achievement of Objectives

Table 8.1: Objective achievement

Objective	Implementation / evidence	Achievement
1. Collect and prepare Binance 15m data	317,838 clean candles from 2017 to 2026; 643 missing candles reported and segmented	Achieved
2. Build features and 64-candle windows	49 backward-looking features; 312,025 usable windows	Achieved
3. Build leakage-safe targets and split	Six horizons; chronological split with 70-sample purge; 24 of 24 audit checks pass	Achieved
4. Train a GRU for return, direction and regime	3-seed ensemble with three heads; 147,744 parameters per member	Achieved
5. Evaluate against baselines and report weaknesses	Regime +0.15 over baseline; direction macro-F1 about 0.42 vs 0.23; return head worse than the mean	Achieved
6. Dashboard with market, prediction and track record	React dashboard (simple and advanced views), terminal dashboard, FastAPI, SQLite store	Achieved
7. Decision support only, not automatic trading	No order execution, sizing or backtest; limits stated in the dashboard	Achieved
Proposal: compare feedforward, LSTM and GRU models	Only the GRU was evaluated; comparison was made against naive baselines. No feedforward model is in the final code	Not completed
Proposal: forecast the next closing price	Replaced by return, direction and regime targets after early experiments	Changed

8.3 Main Contributions

- A reproducible, leakage-audited data pipeline for 15-minute Bitcoin data with gap-safe windows and targets.
- A multi-task GRU with direction, return and regime outputs, and a result that shows clearly which of the three outputs carries information.
- A dashboard that reports its own running accuracy and shows the baseline next to every number.

8.4 Limitations

- The project uses one asset, one interval and one exchange (BTCUSDT 15-minute on Binance spot).
- The test period is a single market period, and its class balance differs from the training period.
- The direction head is biased towards NEUTRAL and the return head has no skill.
- There is no backtest, so fees, slippage, funding and position sizing are not modelled.
- Only past price and volume data are used; news, sentiment and macroeconomic data are not included.
- The planned comparison with feedforward and LSTM models was not completed in the final results.
- The live scorecard is a small sample and cannot yet be used to judge the model.
- The system is a research helper. It has no authentication, no scheduler and no deployment service.

8.5 Future Work

- Test a different NEUTRAL band or a different direction target, since the NEUTRAL bias is the main weakness of the direction head.
- Complete the model comparison with LSTM and feedforward baselines under the same split and metrics.
- Build a backtest that includes fees and slippage before any claim about usefulness in trading.

- Test other assets and intervals, and retrain the model on a schedule as the market changes.
- Add other information sources such as news or sentiment, which were part of the original discussion of Bitcoin price drivers.
- Remove or replace the return head, because it showed no skill.
- Collect live results over a longer period to measure the dashboard's real hit rate.

References

[1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1724–1734.

[3] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” arXiv:1412.3555, 2014.

[4] M. Liu, G. Li, J. Li, X. Zhu, and Y. Yao, “Forecasting the price of Bitcoin using deep learning,” Finance Research Letters, vol. 40, Art. no. 101755, 2021.

[5] T. I. Adegboruwa, S. A. Adeshina, and M. M. Boukar, “Time series analysis and prediction of Bitcoin using long short-term memory neural network,” in Proc. 15th Int. Conf. Electronics, Computer and Computation (ICECCO), 2019, pp. 1–5.

[6] R. Cont, “Empirical properties of asset returns: stylized facts and statistical issues,” Quantitative Finance, vol. 1, no. 2, pp. 223–236, 2001.

[7] M. López de Prado, Advances in Financial Machine Learning. Hoboken, NJ, USA: Wiley, 2018.

[8] Binance, “Spot API documentation: Kline/candlestick data,” Binance Developers, 2026. [Online]. Available: https://developers.binance.com/docs/binance-spot-api-docs/rest-api/market-data-endpoints

[9] CoinMarketCap, “CoinMarketCap API documentation,” CoinMarketCap Developers, 2026. [Online]. Available: https://coinmarketcap.com/api/documentation/v1/

[10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[11] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**DMNWK.Ranasinghe**	**Registration No.**	**D-BIT-24-0054**
Degree / Intake	**BSc (Hons) in Information Technology** **Intake 41**	**Group No.**	**08**
Project Title	**1**		

Technical Contributions:

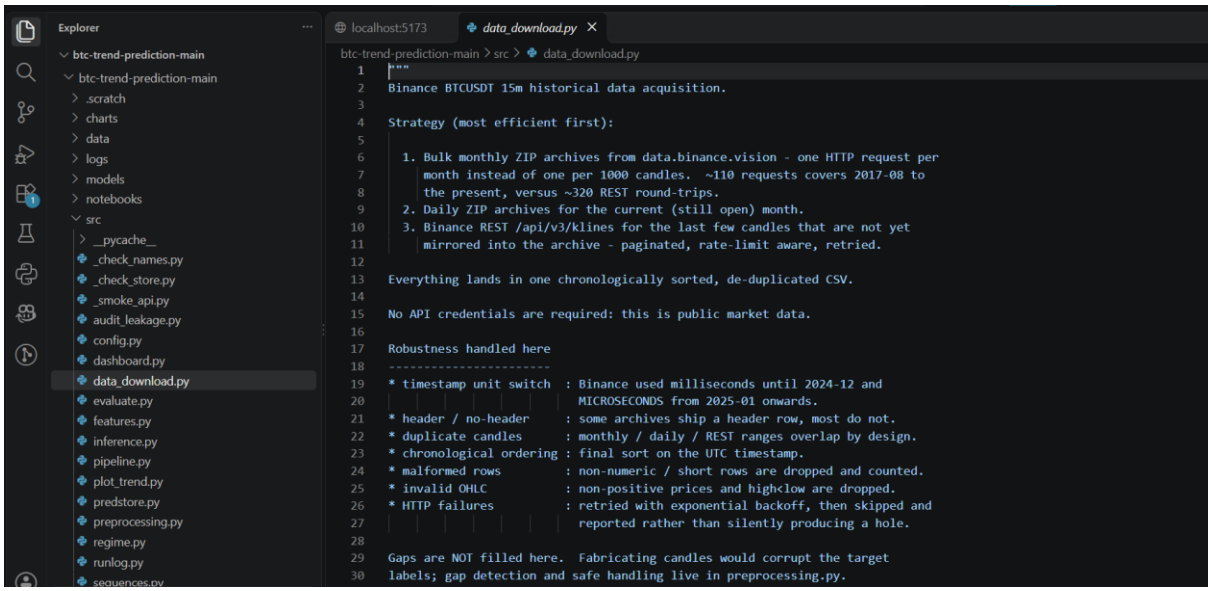
Task / Component	**My Contribution**	**Tools / AI Technique Used**
Historical data collection	Collected BTCUSDT 15-minute historical candles using Binance monthly/daily archives and the REST API; handled pagination, overlapping ranges and chronological ordering.	• Python • Binance market-data archives • REST AP
Data cleaning and validation	Checked timestamp order, duplicate candles, numeric values and OHLC consistency; detected missing 15-minute intervals.	• Python, Pandas • timestamp/OHLC validation

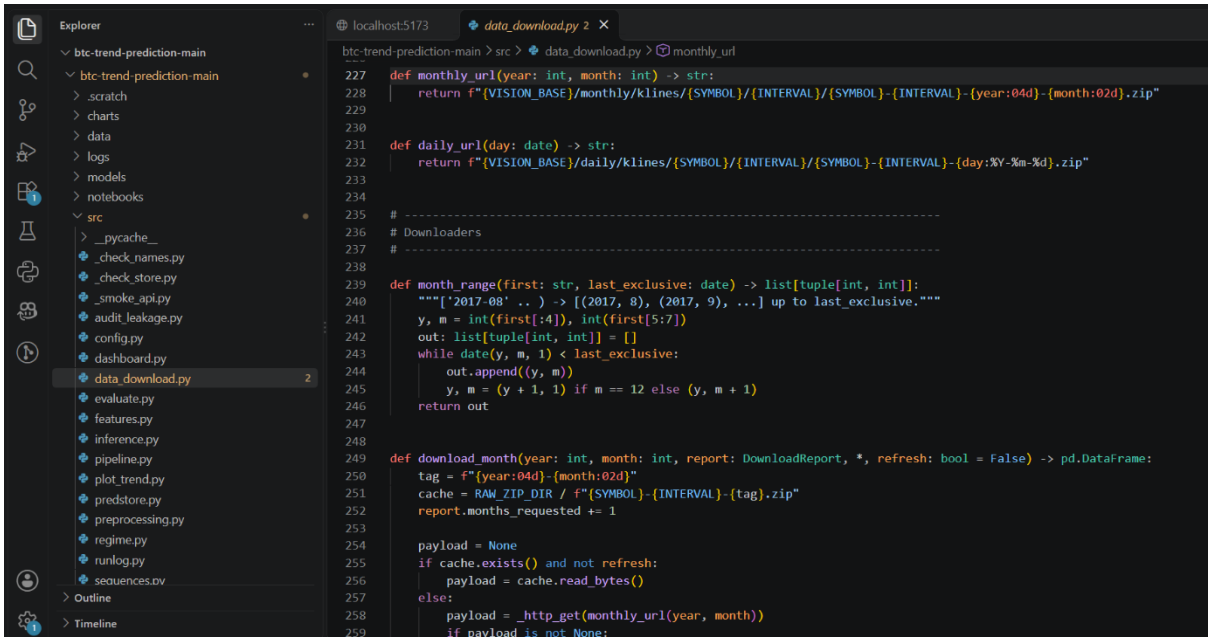
Missing-data handling and segmentation	Kept missing intervals as gaps instead of interpolating synthetic candles; segmented the time series so sequences do not cross gaps.	• Python • gap detection • time-series segmentation
Feature engineering	Prepared historical-only market features such as returns, rolling volatility, RSI, Bollinger-style measures, volume activity and ATR percentage; avoided future-looking inputs.	• Python, Pandas • technical indicators • rolling-window features
Leakage prevention and target separation	Kept future-return targets separate from input features; used training-only thresholds and chronological splits to reduce information leakage.	• Time-series validation • leakage audit • chronological splitting
Dataset and sequence preparation	Prepared valid 64-candle lookback windows, removed feature warm-up rows, used chronological train/validation/test splits and purge buffers around split boundaries.	• Python, NumPy • sequence construction • train-only scaling

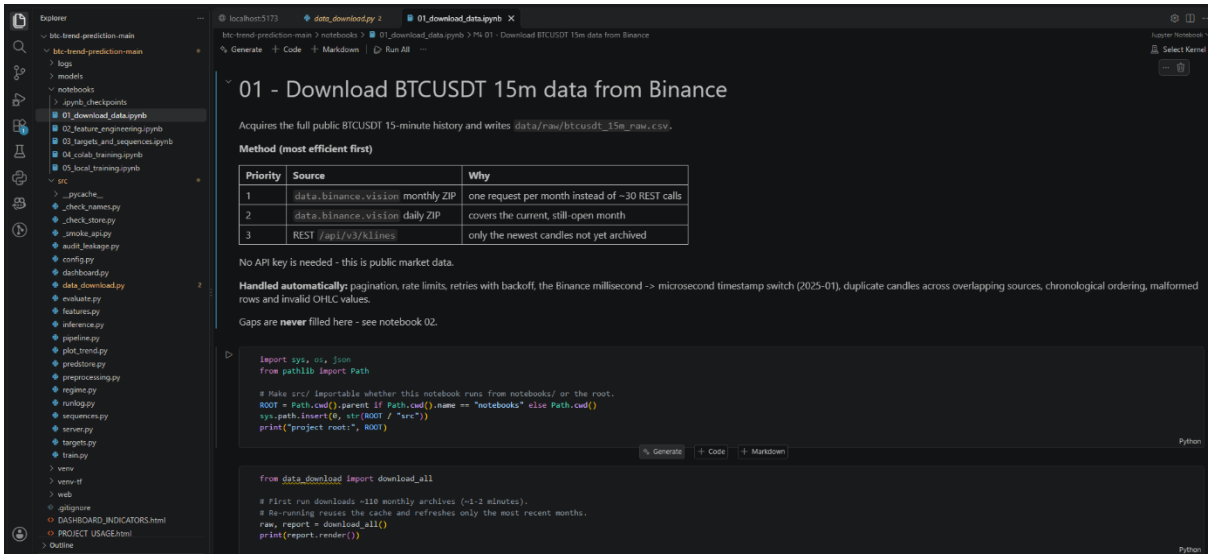
Processing Pipeline Execution Report	Reviewed and documented the end-to-end data-processing workflow, including cleaning, missing-interval checks, feature generation, target preparation, and sequence generation.	• Python • Data-processing pipeline • Pipeline logging • Data validation • Time-series preprocessing

Evidence / Output

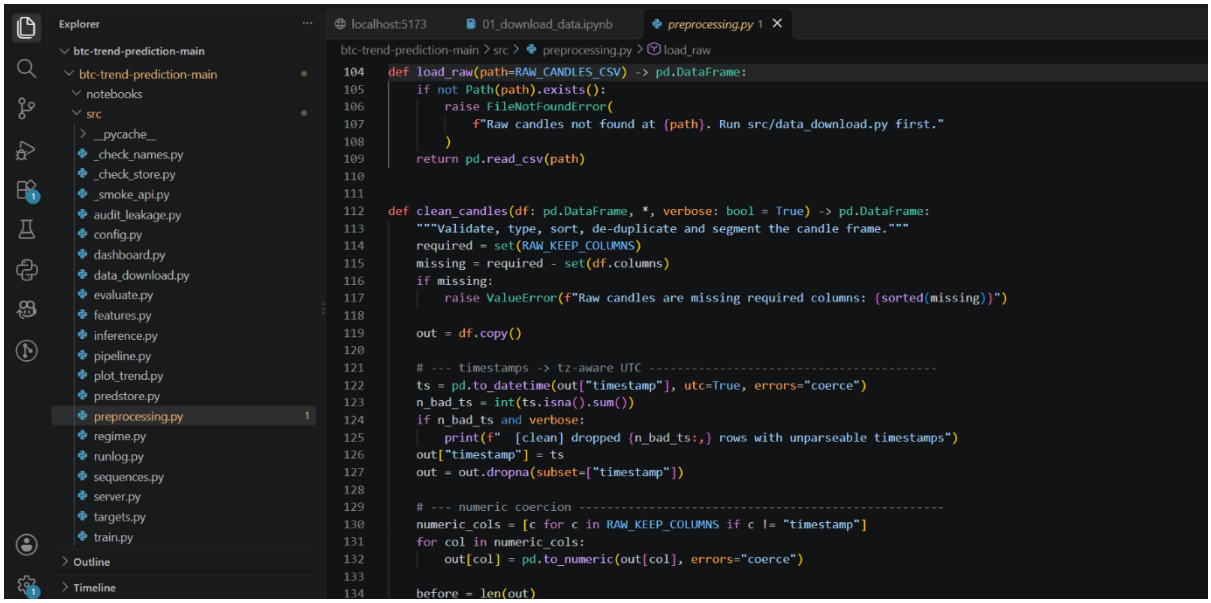
Historical data collection

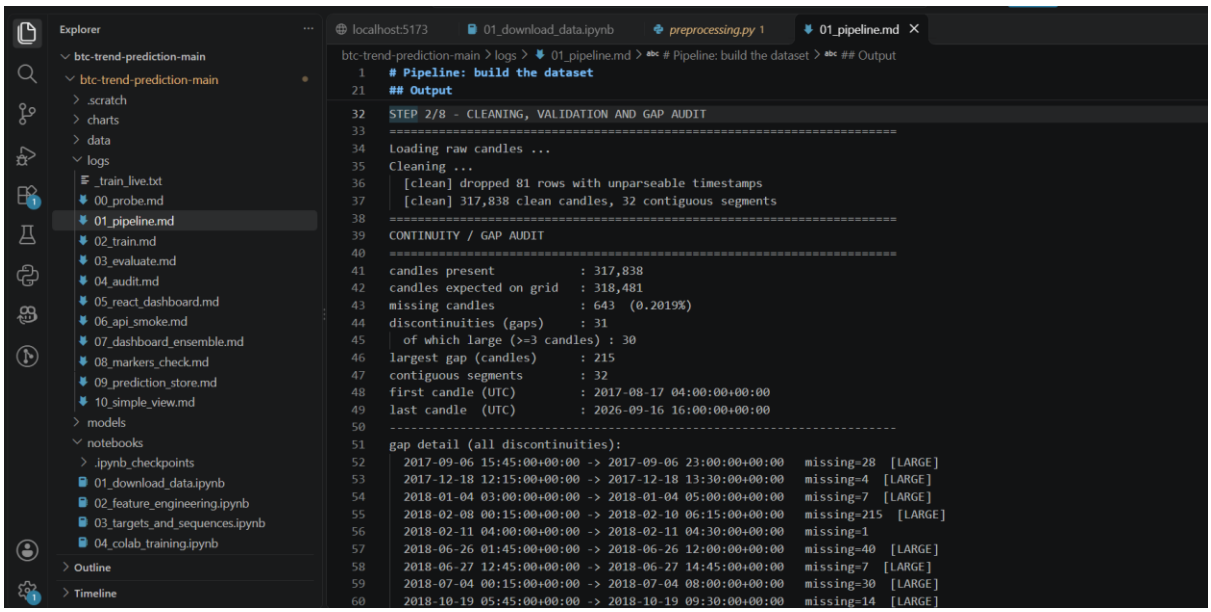


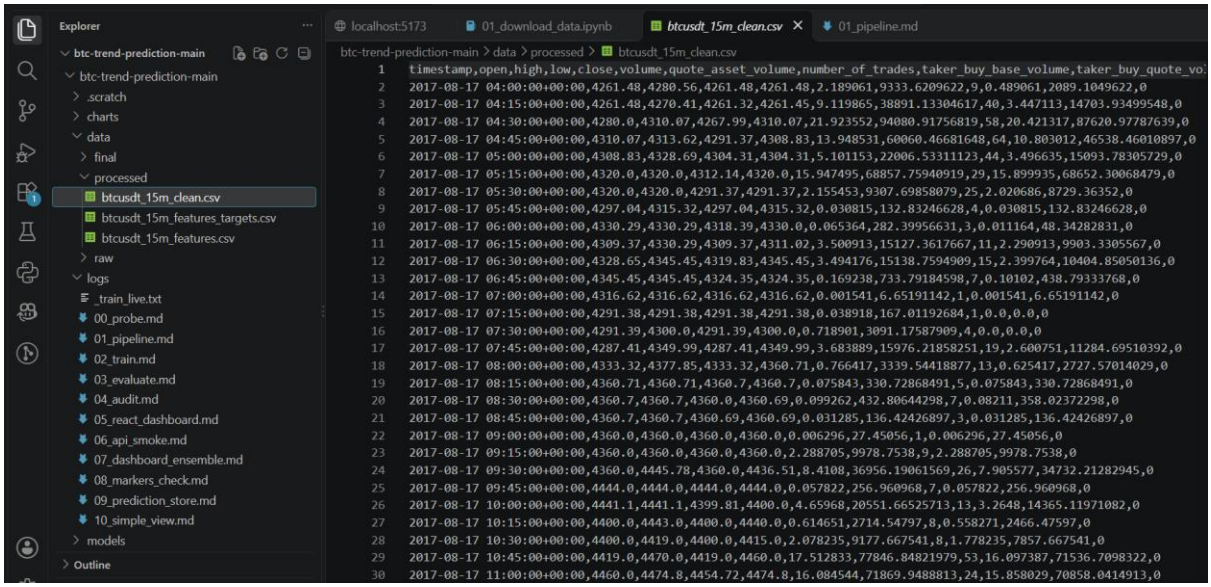




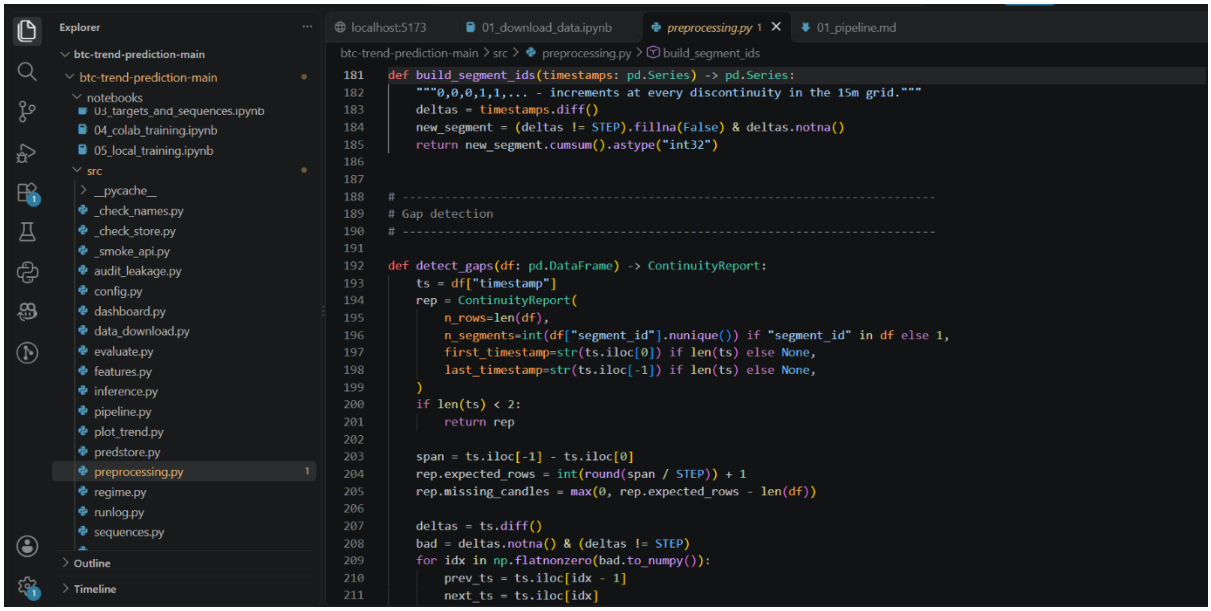
Data cleaning and validation

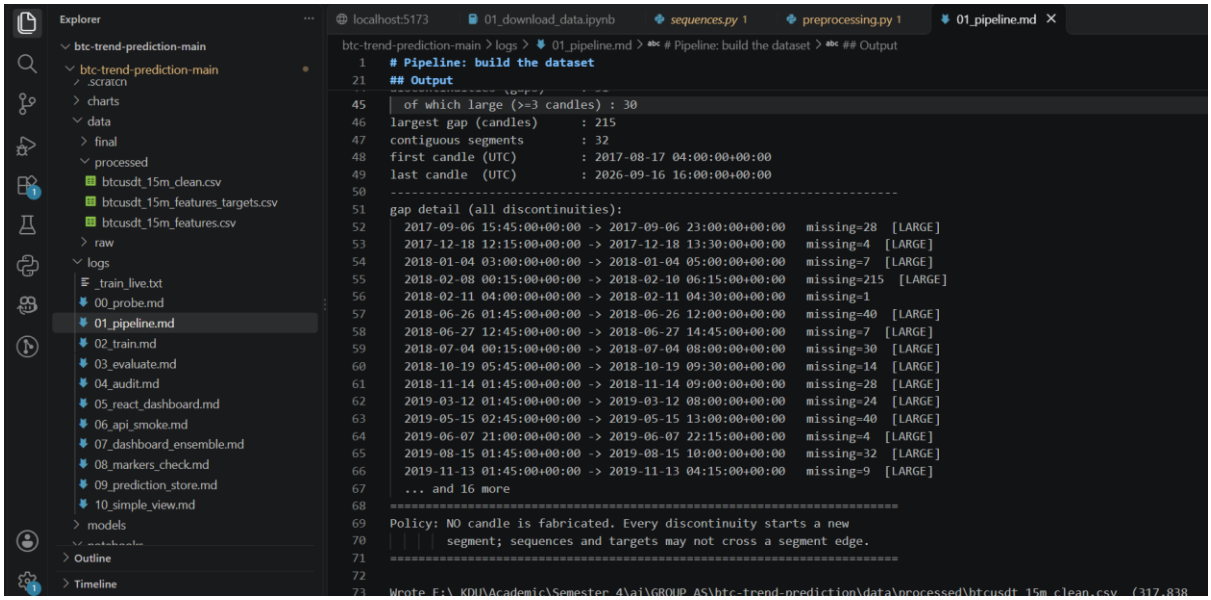

Explorer
btc-trend-prediction-main
btc-trend-prediction-main
notebooks
src
__pycache__
_check_names.py
_check_store.py
_smoke_api.py
audit_leakage.py
config.py
dashboard.py
data_download.py
evaluate.py
features.py
inference.py
pipeline.py
plot_trend.py
predstore.py
preprocessing.py
regime.py
runlog.py
sequences.py
server.py
targets.py
train.py
Outline
Timeline
localhost:5173
01_download_data.ipynb
preprocessing.py 1
btc-trend-prediction-main > src > preprocessing.py > load_raw
104 def load_raw(path=RAW_CANDLES_CSV) -> pd.DataFrame:
105 if not Path(path).exists():
106 raise FileNotFoundError(
107 f"Raw candles not found at {path}. Run src/data_download.py first."
108)
109 return pd.read_csv(path)
110
111
112 def clean_candles(df: pd.DataFrame, *, verbose: bool = True) -> pd.DataFrame:
113 """Validate, type, sort, de-duplicate and segment the candle frame."""
114 required = set(RAW_KEEP_COLUMNS)
115 missing = required - set(df.columns)
116 if missing:
117 raise ValueError(f"Raw candles are missing required columns: {sorted(missing)}")
118
119 out = df.copy()
120
121 # --- timestamps -> tz-aware UTC ----------
122 ts = pd.to_datetime(out["timestamp"], utc=True, errors="coerce")
123 n_bad_ts = int(ts.isna().sum())
124 if n_bad_ts and verbose:
125 print(f" [clean] dropped {n_bad_ts:,} rows with unparseable timestamps")
126 out["timestamp"] = ts
127 out = out.dropna(subset=["timestamp"])
128
129 # --- numeric coercion ----------
130 numeric_cols = [c for c in RAW_KEEP_COLUMNS if c != "timestamp"]
131 for col in numeric_cols:
132 out[col] = pd.to_numeric(out[col], errors="coerce")
133
134 before = len(out)


Explorer
btc-trend-prediction-main
btc-trend-prediction-main
.scratch
charts
data
logs
_train_live.txt
00_probe.md
01_pipeline.md
02_train.md
03_evaluate.md
04_audit.md
05_react_dashboard.md
06_api_smoke.md
07_dashboard_ensemble.md
08_markers_check.md
09_prediction_store.md
10_simple_view.md
models
notebooks
.ipynb_checkpoints
01_download_data.ipynb
02_feature_engineering.ipynb
03_targets_and_sequences.ipynb
04_colab_training.ipynb
Outline
Timeline
localhost:5173
01_download_data.ipynb
preprocessing.py 1
01_pipeline.md
btc-trend-prediction-main > logs > 01_pipeline.md > # Pipeline: build the dataset > ## Output
1 # Pipeline: build the dataset
21 ## Output
32 STEP 2/8 - CLEANING, VALIDATION AND GAP AUDIT
33 ==
34 Loading raw candles ...
35 Cleaning ...
36 [clean] dropped 81 rows with unparseable timestamps
37 [clean] 317,838 clean candles, 32 contiguous segments
38 ==
39 CONTINUITY / GAP AUDIT
40 ==
41 candles present : 317,838
42 candles expected on grid : 318,481
43 missing candles : 643 (0.2019%)
44 discontinuities (gaps) : 31
45 of which large (>=3 candles) : 30
46 largest gap (candles) : 215
47 contiguous segments : 32
48 first candle (UTC) : 2017-08-17 04:00:00+00:00
49 last candle (UTC) : 2026-09-16 16:00:00+00:00
50 ----------
51 gap detail (all discontinuities):
52 2017-09-06 15:45:00+00:00 -> 2017-09-06 23:00:00+00:00 missing=28 [LARGE]
53 2017-12-18 12:15:00+00:00 -> 2017-12-18 13:30:00+00:00 missing=4 [LARGE]
54 2018-01-04 03:00:00+00:00 -> 2018-01-04 05:00:00+00:00 missing=7 [LARGE]
55 2018-02-08 00:15:00+00:00 -> 2018-02-10 06:15:00+00:00 missing=215 [LARGE]
56 2018-02-11 04:00:00+00:00 -> 2018-02-11 04:30:00+00:00 missing=1
57 2018-06-26 01:45:00+00:00 -> 2018-06-26 12:00:00+00:00 missing=40 [LARGE]
58 2018-06-27 12:45:00+00:00 -> 2018-06-27 14:45:00+00:00 missing=7 [LARGE]
59 2018-07-04 00:15:00+00:00 -> 2018-07-04 08:00:00+00:00 missing=30 [LARGE]
60 2018-10-19 05:45:00+00:00 -> 2018-10-19 09:30:00+00:00 missing=14 [LARGE]

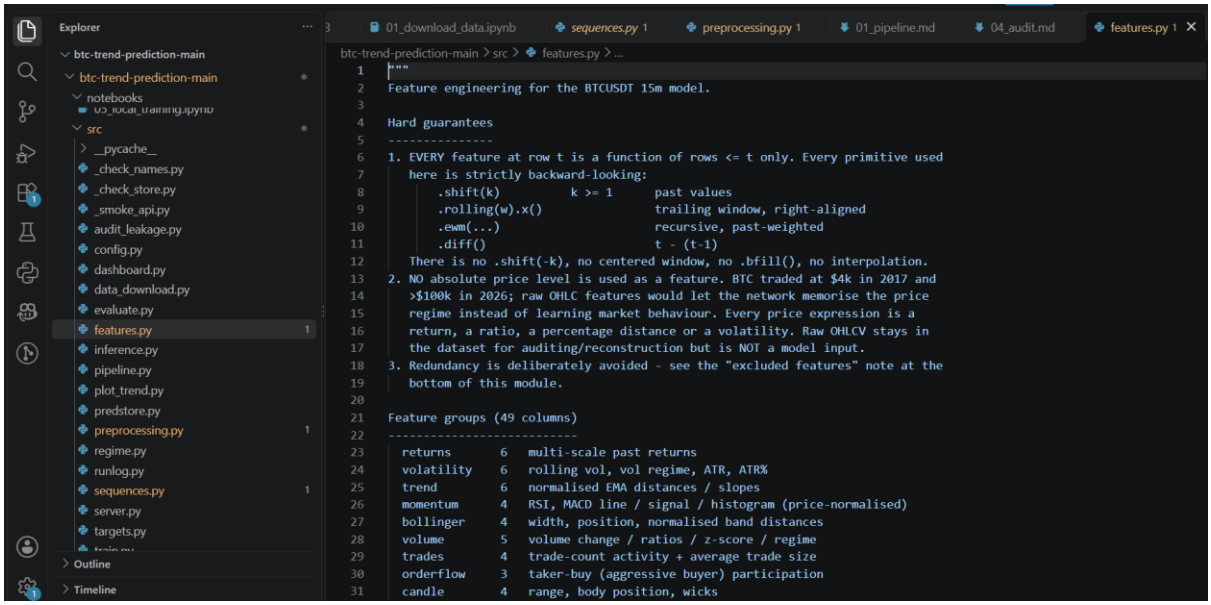


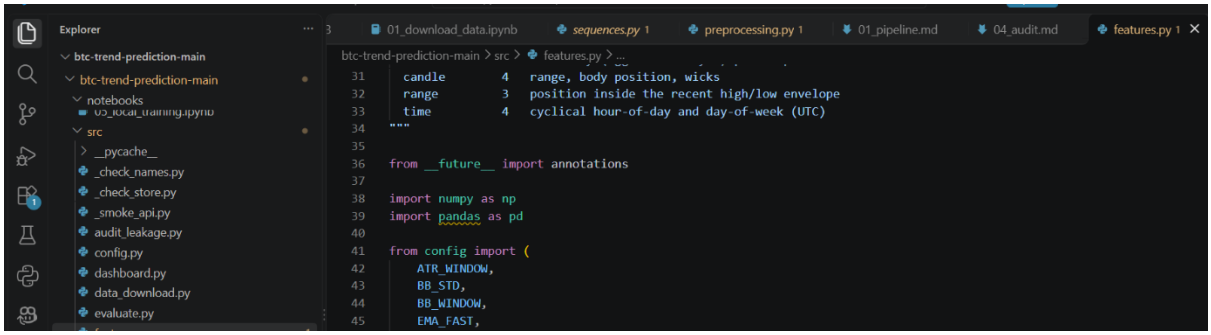
Missing-Data Handling and Segmentation

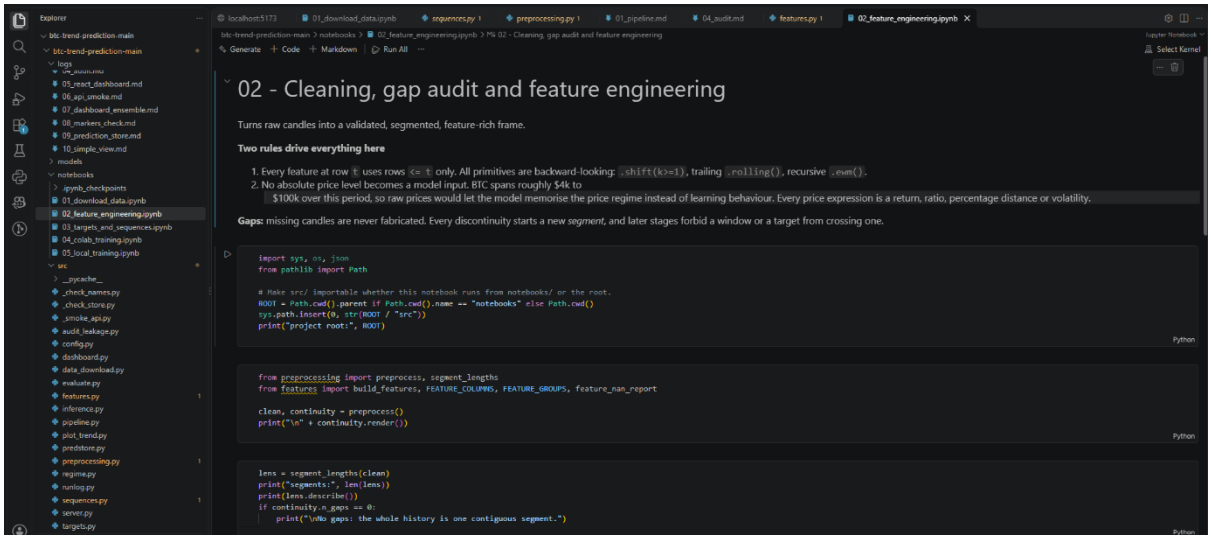


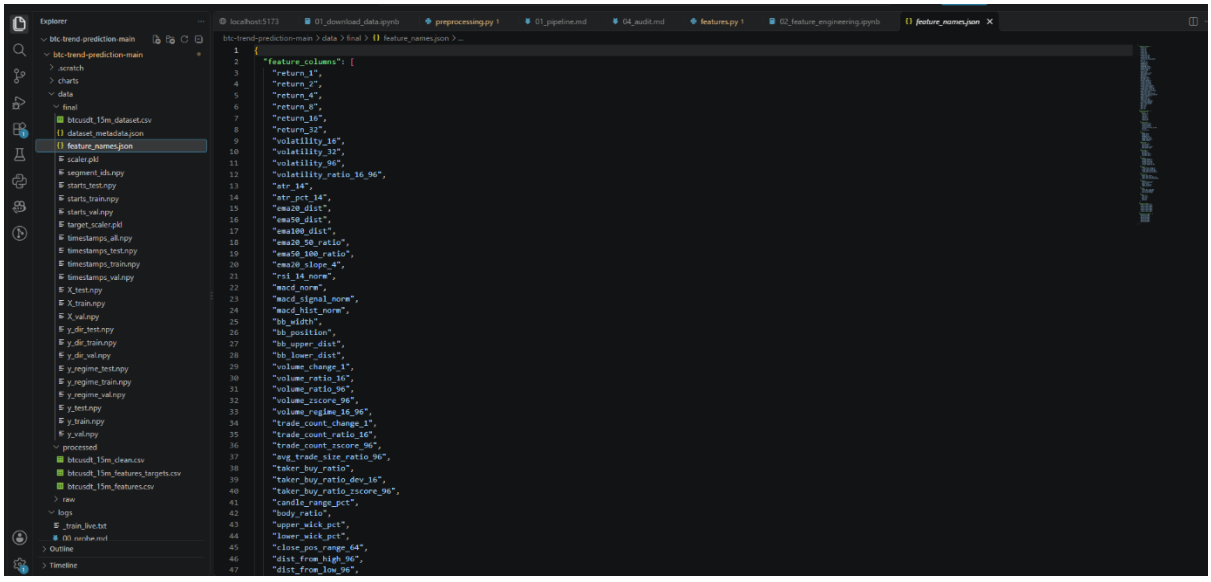


Feature engineering

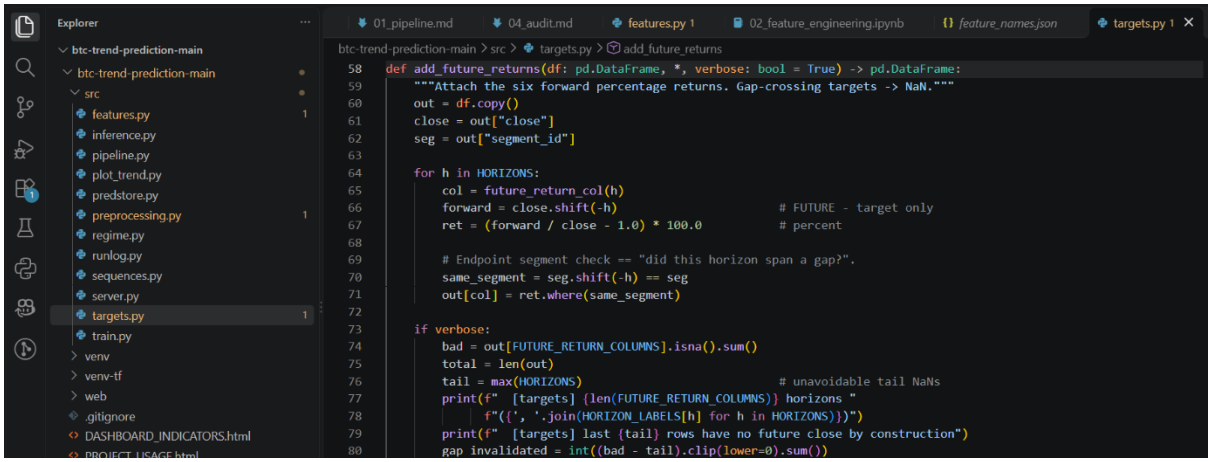


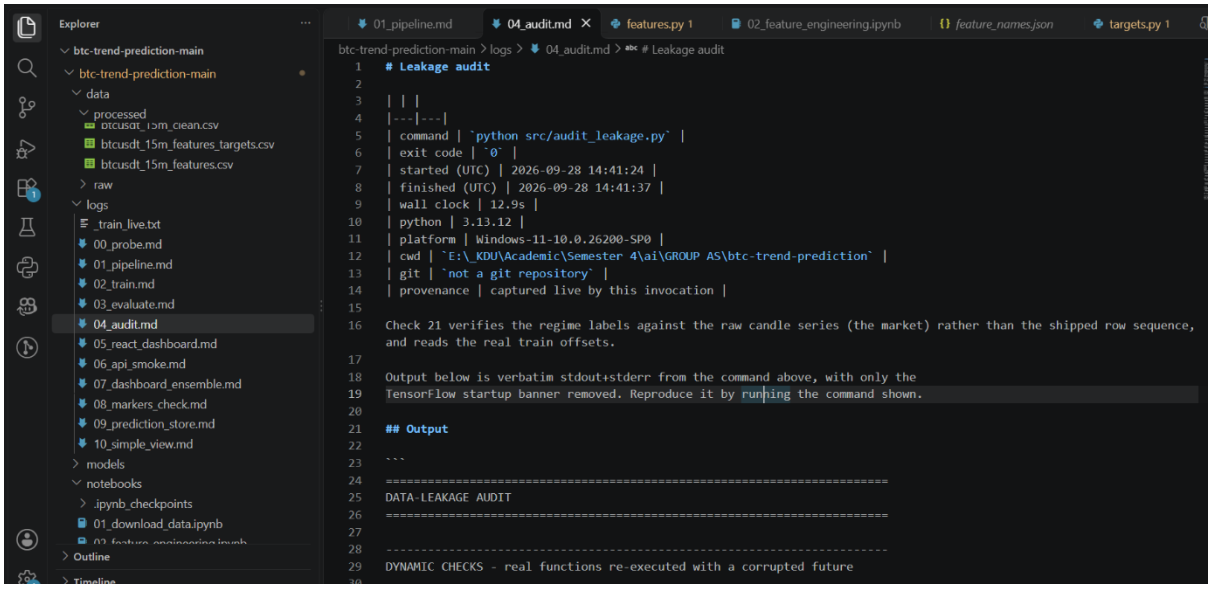

Explorer
btc-trend-prediction-main
notebooks
src
__pycache__
_check_names.py
_check_store.py
_smoke_api.py
audit_leakage.py
config.py
dashboard.py
data_download.py
evaluate.py
01_download_data.ipynb
sequences.py 1
preprocessing.py 1
01_pipeline.md
04_audit.md
features.py 1
candle 4 range, body position, wicks
range 3 position inside the recent high/low envelope
time 4 cyclical hour-of-day and day-of-week (UTC)
"""
from __future__ import annotations
import numpy as np
import pandas as pd
from config import (
ATR_WINDOW,
BB_STD,
BB_WINDOW,
EMA_FAST,

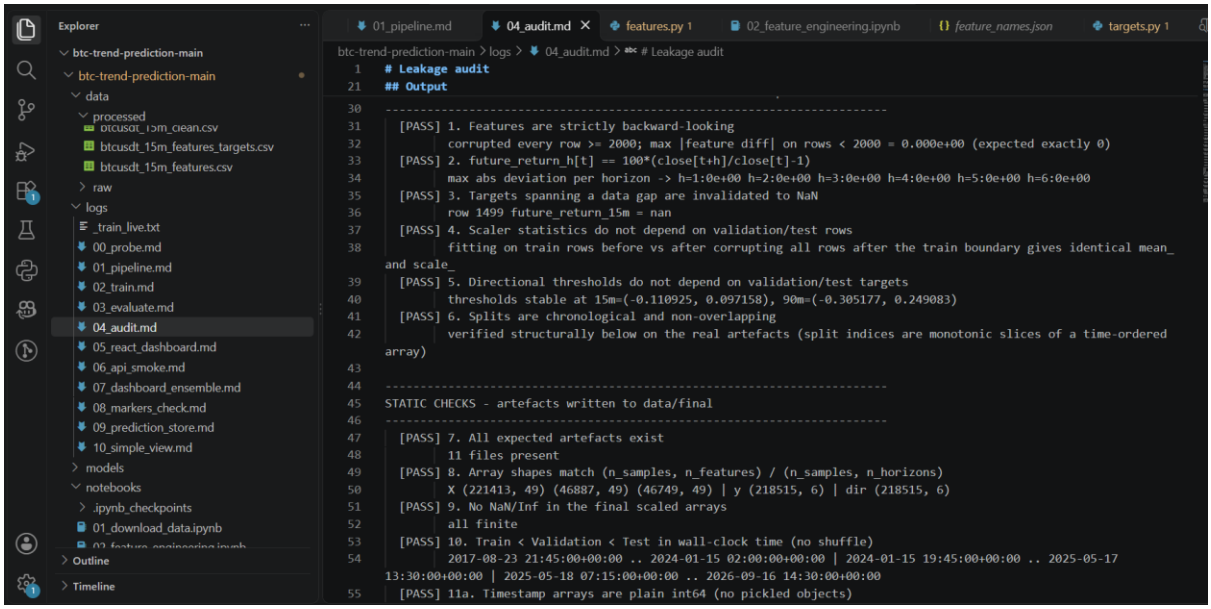

02_feature_engineering.ipynb
02 - Cleaning, gap audit and feature engineering
Turns raw candles into a validated, segmented, feature-rich frame.
Two rules drive everything here
1. Every feature at row t uses rows <= t only. All primitives are backward-looking: .shift(k>=1), trailing .rolling(), recursive .ewm().
2. No absolute price level becomes a model input. BTC spans roughly $4k to $100k over this period, so raw prices would let the model memorise the price regime instead of learning behaviour. Every price expression is a return, ratio, percentage distance or volatility.
Gaps: missing candles are never fabricated. Every discontinuity starts a new segment, and later stages forbid a window or a target from crossing one.
import sys, os, json
from pathlib import Path
Make src/ importable whether this notebook runs from notebooks/ or the root.
ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
print("project root:", ROOT)
Python
from preprocessing import preprocess, segment_lengths
from features import build_features, FEATURE_COLUMNS, FEATURE_GROUPS, feature_nan_report
clean, continuity = preprocess()
print("\n" + continuity.render())
Python
lens = segment_lengths(clean)
print("segments:", len(lens))
print(lens.describe())
if continuity.n_gaps == 0:
print("\nNo gaps: the whole history is one contiguous segment.")
Python


feature_names.json
{
"feature_columns": [
"return_1",
"return_2",
"return_4",
"return_8",
"return_16",
"return_32",
"volatility_16",
"volatility_32",
"volatility_96",
"volatility_ratio_16_96",
"atr_14",
"atr_pct_14",
"ema20_dist",
"ema50_dist",
"ema100_dist",
"ema20_50_ratio",
"ema50_100_ratio",
"ema20_slope_4",
"rsi_14_norm",
"macd_norm",
"macd_signal_norm",
"macd_hist_norm",
"bb_width",
"bb_position",
"bb_upper_dist",
"bb_lower_dist",
"volume_change_1",
"volume_ratio_16",
"volume_ratio_96",
"volume_zscore_96",
"volume_regime_16_96",
"trade_count_change_1",
"trade_count_ratio_16",
"trade_count_zscore_96",
"avg_trade_size_ratio_96",
"taker_buy_ratio",
"taker_buy_ratio_dev_16",
"taker_buy_ratio_zscore_96",
"candle_range_pct",
"body_ratio",
"upper_wick_pct",
"lower_wick_pct",
"close_pos_range_64",
"dist_from_high_96",
"dist_from_low_96",

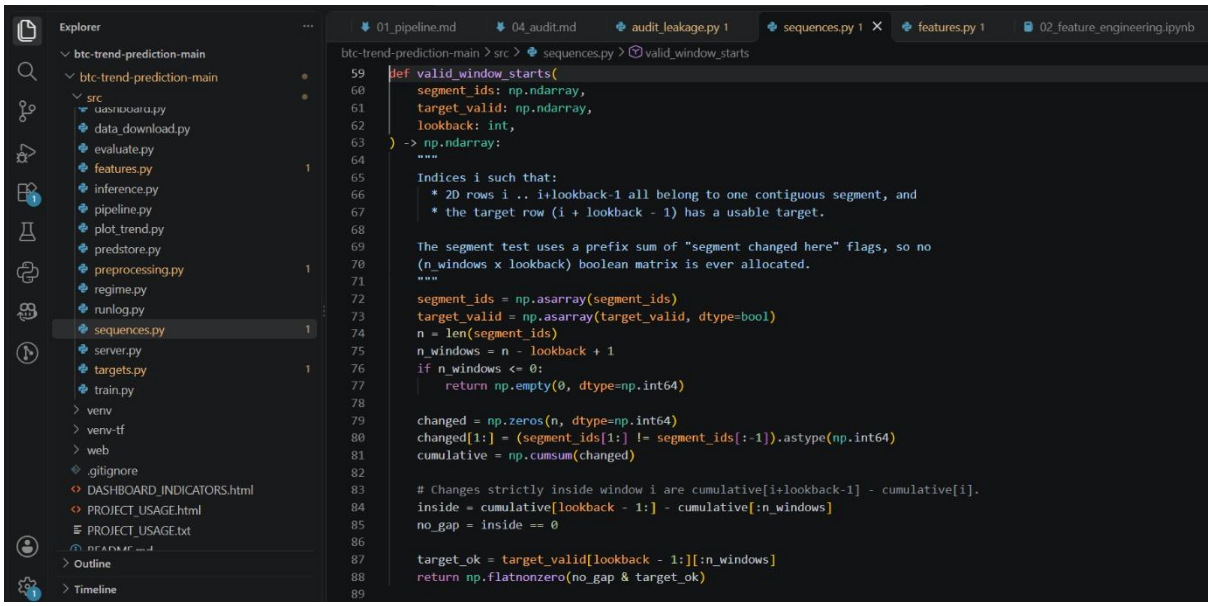
Leakage Prevention and Target Separation



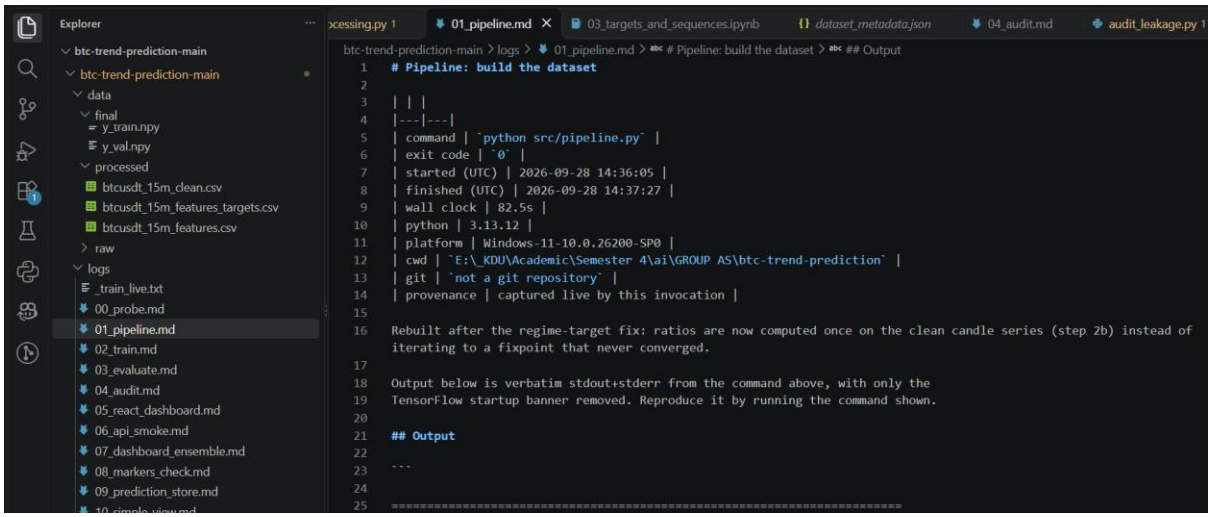




Dataset and Sequence Preparation



Processing Pipeline Execution Report



Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 17.10.2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**PYB Karunasena**	**Registration No.**	**D-BIS-24-0016**
Degree / Intake	**BSc (Hons) in Information Technology** **Intake 41**	**Group No.**	**08**
Project Title	**CryptoForecast - Bitcoin Trend Prediction Using Deep Learning Time-Series Analysis**		

Technical Contributions:

Task / Component	**My Contribution**	**Tools / AI Technique Used**	**Evidence / Output**
Model evaluation	Evaluate the trained BTC price-prediction model on the validation and test splits across the 15, 30, 45, 60, 75 and 90-minute horizons. Compare regression and direction predictions with the project's naive baselines, and review per-class precision, recall, F1, confusion matrices and confidence calibration.	Python; NumPy; pandas; TensorFlow/Keras; evaluation metrics implemented in src/evaluate.py	Evaluation tables and metric summaries; optional CSV output from src/evaluate.py. Source: https://github.com/ushanrashmika23/ai-assesment-btc-prediction-sys/blob/main/src/evaluate.py

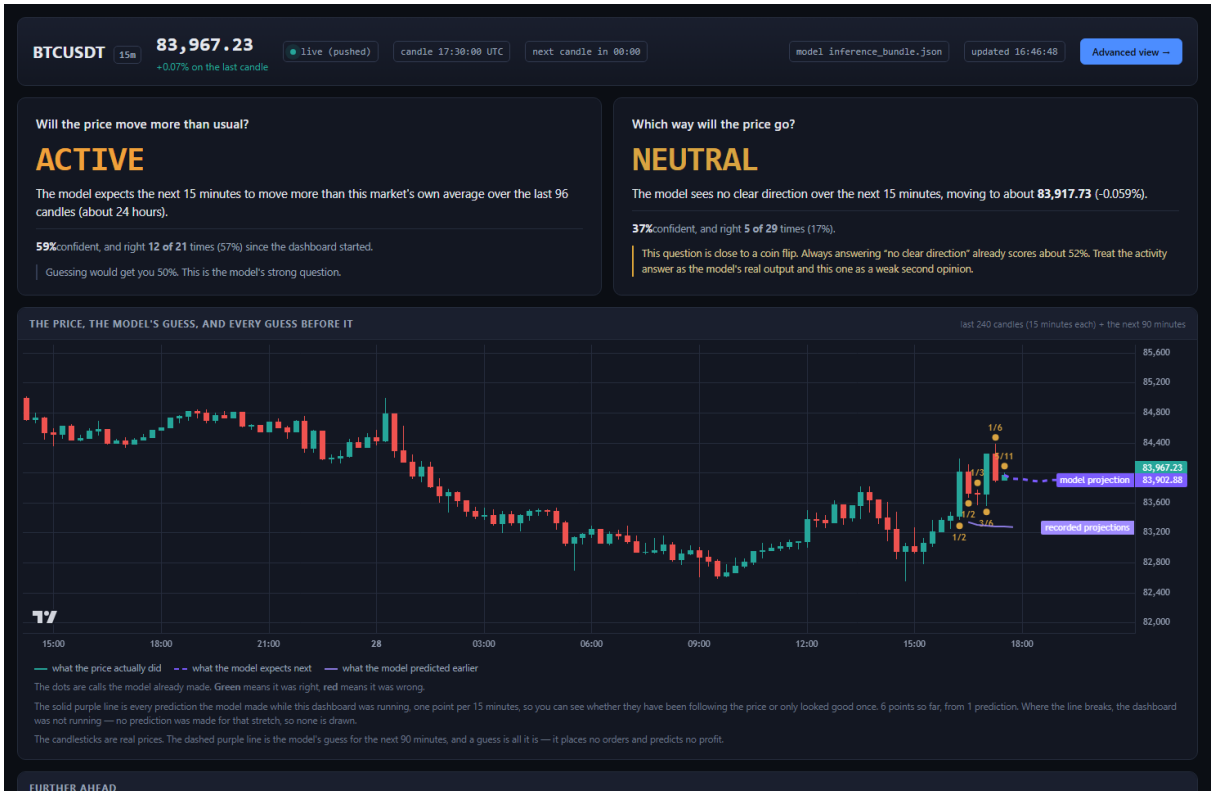
Automated model/data testing	Review and run the automated acceptance tests, checking that features do not use future data, train/validation/test splits remain chronological, windows do not cross market-data gaps, and the data-quality audit passes	pytest; tests/run_tests.py; tests/test_leakage.py; tests/test_split.py; tests/test_windows.py; tests/test_audit.py	Test log and machine-readable results report: tests/logs/test_run.log and tests/logs/results.json; audit transcript. Source: https://github.com/ushanrashmika23/ai-assesment-btc-prediction-sys/tree/main/tests
System and prediction-output testing	Check API responses and prediction payloads, stored prediction records, dashboard rendering, and chart-marker handling. Record failures or skipped tests with their reasons and use the generated evidence to report test outcomes.	Python; pytest; FastAPI TestClient; Node.js checks; tests/test_api.py; tests/test_store.py; tests/test_dashboard.py; tests/test_markers.py	Per-test outcomes and captured screenshots/page dumps under tests/screenshots/. Source: https://github.com/ushanrashmika23/ai-assesment-btc-prediction-sys/tree/main/tests

Screenshot Evidence from the Tests Folder

The following screenshots are saved in the project's tests/screenshots folder. Select a link to open the corresponding screenshot in the GitHub repository.

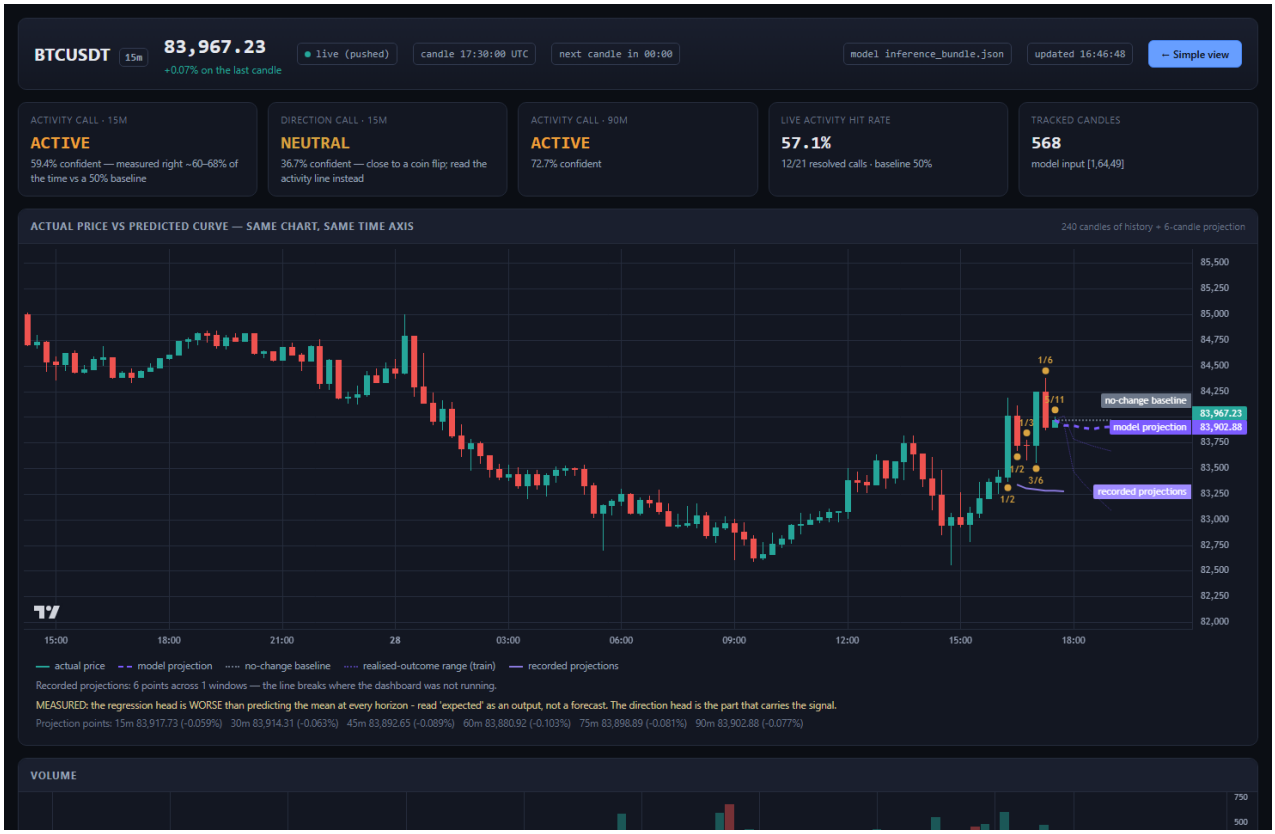
Screenshot 1 – React dashboard (simple/default view)

Evidence of the dashboard's default/simple view.



Screenshot 2 – React dashboard (advanced view)

Evidence of the dashboard after switching to the advanced view.



Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature: YBKarunasena **Date:** 17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	KDU Rashmika	**Registration No.**	D/BSE/25/0001
Degree / Intake	42	**Group No.**	08
Project Title	**BTCUSDT 15-Minute Short-Term Trend Prediction using a Multi-Task GRU Ensemble (Binance data, 15-90 min horizons)**		

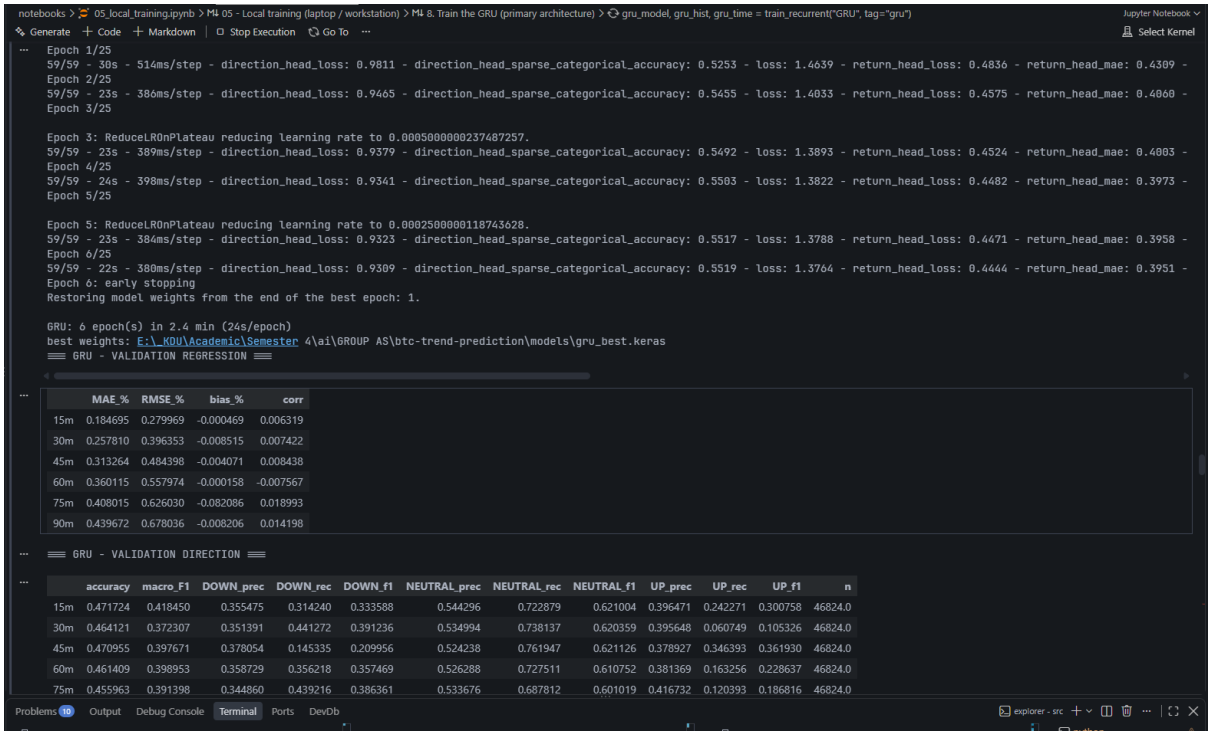
Technical Contributions:

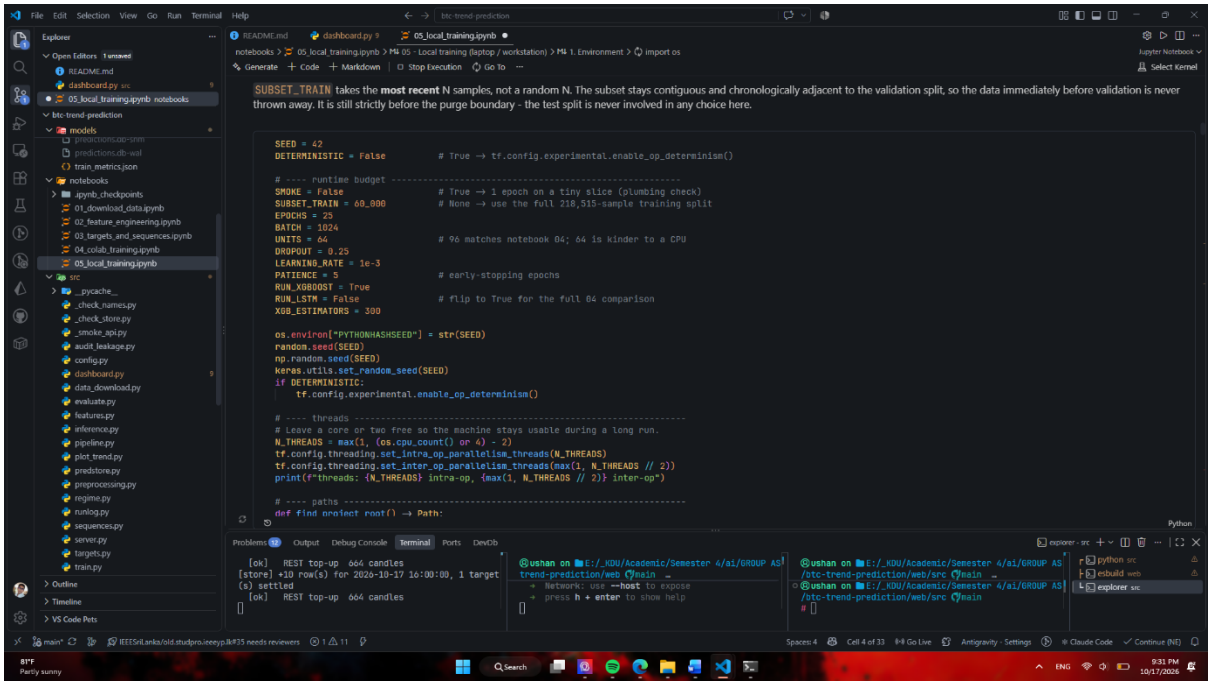
Task / Component	**My Contribution**	**Tools / AI Technique Used**	**Evidence / Output**
1. Model architecture design	Designed the core network in build_multi_task_model() : input window of 64 candles x 49 features (16 h of 15-min BTCUSDT history), two stacked recurrent layers (64 units, return_sequences=True then False) Made the recurrent cell switchable (GRU or	Python, TensorFlow 2.21 Keras 3.14 GRU LSTM	src/train.py (build_multi_task_model); models/*.keras; architecture settings in src/config.py

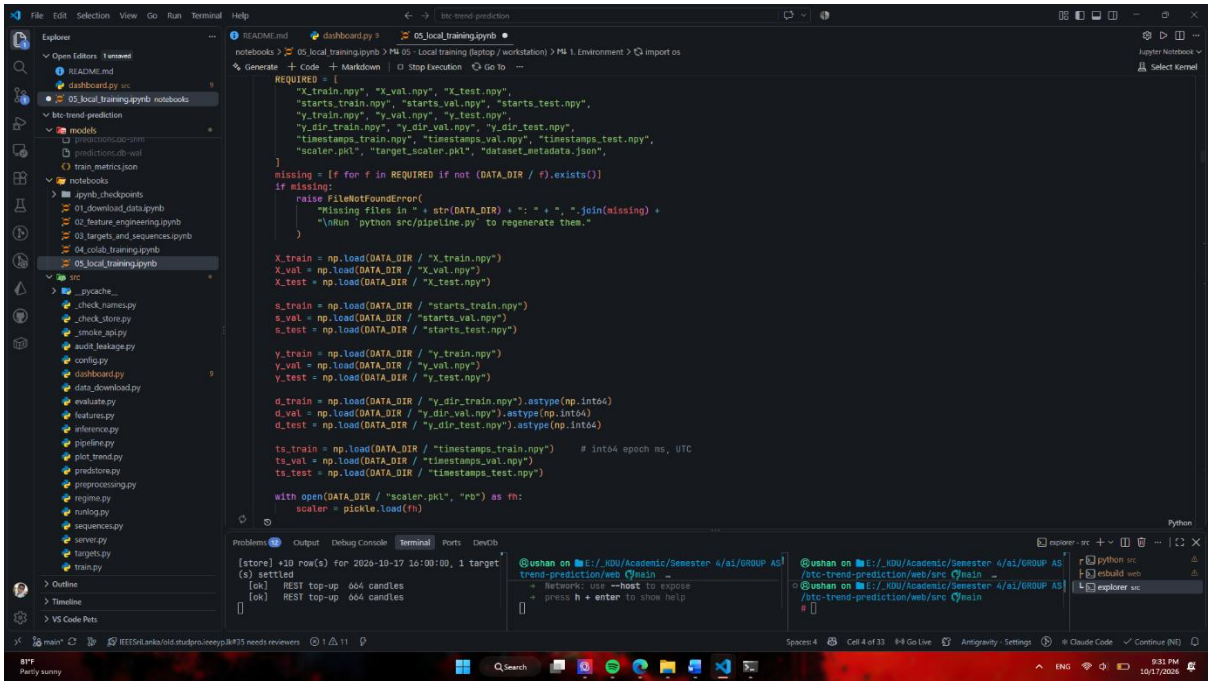
	LSTM) so both could be trained and compared fairly on the same data; GRU was chosen as the primary model.		
2. Multi-task output heads and loss design	Built three heads on the shared trunk, return head (6-horizon regression on scaled returns, MSE), direction head (DOWN/NEUTRAL/UP for 15-90 min) and regime head (QUIET/ACTIVE volatility for 15, 30, 60, 90 min). Fixed a softmax bug: softmax is now applied after Reshape((6,3)) so each horizon's probabilities sum to 1. Added a value-level assertion in the build step to catch it. Wrote a custom weighted sparse cross-entropy loss with train-only class weights (strength 0.5) to reduce the direction head's NEUTRAL bias.	Multi-task learning custom Keras loss class-weighted cross-entropy MSE	make_weighted_scc() and compute_direction_class_weights( ) in src/train.py; self-check in the model builder

3. Training pipeline and model selection	Implemented headless training (python src/train.py) with CLI options and a streaming PyDataset that feeds windows via the project's own SequenceBatcher, so training and inference share the same windowing. Wrote SelectionScoreCallback: each epoch it scores the validation split and combines direction macro-F1 and regime accuracy (0.5/0.5) into a selection score. This drives EarlyStopping (patience 5), ReduceLROnPlateau (x0.5) and ModelCheckpoint, instead of val_loss, which is dominated by the regression head (no predictive skill). Training settings: Adam, lr 1e-3, batch 1024, up to 25 epochs.	Keras callbacks PyDataset scikit-learn Adam optimiser	src/train.py; logs/02_train.md (verbatim training transcript); notebooks/04_colab_training.ipynb and 05_local_training.ipynb

4. Seed ensembling and model packaging	Trained three seeds (42, 43, 44) and averaged their softmax outputs rather than picking the best seed, because the seed-to-seed gaps were within noise. Packaged the members into one inference bundle for the dashboard. Kept the test split untouched during development: architecture and checkpoint selection used validation metrics only.	Ensemble averaging fixed random seeds Keras model serialisation (.keras)	models/gru_s42/43/44_best.keras; models/inference_bundle.json; run: train.py --arch gru --seeds 42,43,44 --epochs 25 --patience 5 --batch 1024


notebooks > 05_local_training.ipynb > M↓ 05 - Local training (laptop / workstation) > M↓ 8. Train the GRU (primary architecture) > gru_model, gru_hist, gru_time = train_recurrent("GRU", tag="gru")
Jupyter Notebook
Generate + Code + Markdown | Stop Execution Go To ...
Select Kernel
Epoch 1/25
59/59 - 30s - 514ms/step - direction_head_loss: 0.9811 - direction_head_sparse_categorical_accuracy: 0.5253 - loss: 1.4639 - return_head_loss: 0.4836 - return_head_mae: 0.4309 -
Epoch 2/25
59/59 - 23s - 386ms/step - direction_head_loss: 0.9465 - direction_head_sparse_categorical_accuracy: 0.5455 - loss: 1.4033 - return_head_loss: 0.4575 - return_head_mae: 0.4060 -
Epoch 3/25
Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
59/59 - 23s - 389ms/step - direction_head_loss: 0.9379 - direction_head_sparse_categorical_accuracy: 0.5492 - loss: 1.3893 - return_head_loss: 0.4524 - return_head_mae: 0.4003 -
Epoch 4/25
59/59 - 24s - 398ms/step - direction_head_loss: 0.9341 - direction_head_sparse_categorical_accuracy: 0.5503 - loss: 1.3822 - return_head_loss: 0.4482 - return_head_mae: 0.3973 -
Epoch 5/25
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
59/59 - 23s - 384ms/step - direction_head_loss: 0.9323 - direction_head_sparse_categorical_accuracy: 0.5517 - loss: 1.3788 - return_head_loss: 0.4471 - return_head_mae: 0.3958 -
Epoch 6/25
59/59 - 22s - 380ms/step - direction_head_loss: 0.9309 - direction_head_sparse_categorical_accuracy: 0.5519 - loss: 1.3764 - return_head_loss: 0.4444 - return_head_mae: 0.3951 -
Epoch 6: early stopping
Restoring model weights from the end of the best epoch: 1.
GRU: 6 epoch(s) in 2.4 min (24s/epoch)
best weights: E:_KDU\Academic\Semester 4\ai\GROUP AS\btc-trend-prediction\models\gru_best.keras
GRU - VALIDATION REGRESSION
MAE_% RMSE_% bias_% corr
15m 0.184695 0.279969 -0.000469 0.006319
30m 0.257810 0.396353 -0.008515 0.007422
45m 0.313264 0.484398 -0.004071 0.008438
60m 0.360115 0.557974 -0.000158 -0.007567
75m 0.408015 0.626030 -0.082086 0.018993
90m 0.439672 0.678036 -0.008206 0.014198
GRU - VALIDATION DIRECTION
accuracy macro_F1 DOWN_prec DOWN_rec DOWN_f1 NEUTRAL_prec NEUTRAL_rec NEUTRAL_f1 UP_prec UP_rec UP_f1 n
15m 0.471724 0.418450 0.355475 0.314240 0.333588 0.544296 0.722879 0.621004 0.396471 0.242271 0.300758 46824.0
30m 0.464121 0.372307 0.351391 0.441272 0.391236 0.534994 0.738137 0.620359 0.395648 0.060749 0.105326 46824.0
45m 0.470955 0.397671 0.378054 0.145335 0.209956 0.524238 0.761947 0.621126 0.378927 0.346393 0.361930 46824.0
60m 0.461409 0.398953 0.358729 0.356218 0.357469 0.526288 0.727511 0.610752 0.381369 0.163256 0.228637 46824.0
75m 0.455963 0.391398 0.344860 0.439216 0.386361 0.533676 0.687812 0.601019 0.416732 0.120393 0.186816 46824.0
Problems 10 Output Debug Console Terminal Ports DevDb
explorer - src


File Edit Selection View Go Run Terminal Help
btc-trend-prediction
README.md dashboard.py 9 05_local_training.ipynb
notebooks > 05_local_training.ipynb > M↓ 05 - Local training (laptop / workstation) > M↓ 1. Environment > import os
Jupyter Notebook
Generate + Code + Markdown | Stop Execution Go To ...
Select Kernel
SUBSET_TRAIN takes the most recent N samples, not a random N. The subset stays contiguous and chronologically adjacent to the validation split, so the data immediately before validation is never thrown away. It is still strictly before the purge boundary - the test split is never involved in any choice here.
SEED = 42
DETERMINISTIC = False # True → tf.config.experimental.enable_op_determinism()
---- runtime budget ----
SMOKE = False # True → 1 epoch on a tiny slice (plumbing check)
SUBSET_TRAIN = 60_000 # None → use the full 218,515-sample training split
EPOCHS = 25
BATCH = 1024
UNITS = 64 # 96 matches notebook 04; 64 is kinder to a CPU
DROPOUT = 0.25
LEARNING_RATE = 1e-3
PATIENCE = 5 # early-stopping epochs
RUN_XGBOOST = True
RUN_LSTM = False # flip to True for the full 04 comparison
XGB_ESTIMATORS = 300
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
keras.utils.set_random_seed(SEED)
if DETERMINISTIC:
tf.config.experimental.enable_op_determinism()
---- threads ----
Leave a core or two free so the machine stays usable during a long run.
N_THREADS = max(1, (os.cpu_count() or 4) - 2)
tf.config.threading.set_intra_op_parallelism_threads(N_THREADS)
tf.config.threading.set_inter_op_parallelism_threads(max(1, N_THREADS // 2))
print(f"threads: {N_THREADS} intra-op, {max(1, N_THREADS // 2)} inter-op")
---- paths ----
def find_project_root() -> Path:
Python
Problems 12 Output Debug Console Terminal Ports DevDb
explorer - src
[ok] REST top-up 664 candles
[store] +10 row(s) for 2026-10-17 16:00:00, 1 target
(s) settled
[ok] REST top-up 664 candles
@ushan on E:/_KDU/Academic/Semester 4/ai/GROUP AS/btc-trend-prediction/web main
Network: use --host to expose
press h + enter to show help
@ushan on E:/_KDU/Academic/Semester 4/ai/GROUP AS/btc-trend-prediction/web/src main
@ushan on E:/_KDU/Academic/Semester 4/ai/GROUP AS/btc-trend-prediction/web/src main
python src
esbuild web
explorer src
Spaces: 4 Cell 4 of 33 Go Live Antigravity - Settings Claude Code Continue (NE)
81°F Partly sunny
Search
ENG
9:31 PM 10/17/2026


REQUIRED = [
"X_train.npy", "X_val.npy", "X_test.npy",
"starts_train.npy", "starts_val.npy", "starts_test.npy",
"y_train.npy", "y_val.npy", "y_test.npy",
"y_dir_train.npy", "y_dir_val.npy", "y_dir_test.npy",
"timestamps_train.npy", "timestamps_val.npy", "timestamps_test.npy",
"scaler.pkl", "target_scaler.pkl", "dataset_metadata.json",
]
missing = [f for f in REQUIRED if not (DATA_DIR / f).exists()]
if missing:
raise FileNotFoundError(
"Missing files in " + str(DATA_DIR) + ": " + ", ".join(missing) +
"\nRun `python src/pipeline.py` to regenerate them."
)
X_train = np.load(DATA_DIR / "X_train.npy")
X_val = np.load(DATA_DIR / "X_val.npy")
X_test = np.load(DATA_DIR / "X_test.npy")
s_train = np.load(DATA_DIR / "starts_train.npy")
s_val = np.load(DATA_DIR / "starts_val.npy")
s_test = np.load(DATA_DIR / "starts_test.npy")
y_train = np.load(DATA_DIR / "y_train.npy")
y_val = np.load(DATA_DIR / "y_val.npy")
y_test = np.load(DATA_DIR / "y_test.npy")
d_train = np.load(DATA_DIR / "y_dir_train.npy").astype(np.int64)
d_val = np.load(DATA_DIR / "y_dir_val.npy").astype(np.int64)
d_test = np.load(DATA_DIR / "y_dir_test.npy").astype(np.int64)
ts_train = np.load(DATA_DIR / "timestamps_train.npy")
ts_val = np.load(DATA_DIR / "timestamps_val.npy")
ts_test = np.load(DATA_DIR / "timestamps_test.npy")
with open(DATA_DIR / "scaler.pkl", "rb") as fh:
scaler = pickle.load(fh)


t0 = pd.to_datetime(int(ts[0]), unit="ms", utc=True)
t1 = pd.to_datetime(int(ts[-1]), unit="ms", utc=True)
print(f"{name:6} {len(block):>12,} {len(starts):>10,} {t0:%Y-%m-%d} → {t1:%Y-%m-%d}")
n_holes = int((np.diff(s_train) ≠ 1).sum())
print(f"\ntrain offsets contain {n_holes} hole(s) - gap-excluded windows.")
print("That is exactly why windows must be read via starts_*.npy.")
horizon labels : ['15m', '30m', '45m', '60m', '75m', '90m']
horizon candles : [1, 2, 3, 4, 5, 6] → HORIZON_MAX = 6
lookback=64 features=49 horizons=['15m', '30m', '45m', '60m', '75m', '90m']
split block rows samples target period
train 221,413 218,515 2017-08-23 → 2024-01-15
val 46,887 46,824 2024-01-15 → 2025-05-17
test 46,749 46,686 2025-05-18 → 2026-09-16
train offsets contain 43 hole(s) - gap-excluded windows.
That is exactly why windows must be read via starts_*.npy.

```
print("\nNAIVE DIRECTION (majority train class per horizon)")
display(base_dir)
```

NAIVE REGRESSION (predict 0% return)

	MAE_%	RMSE_%	bias_%	corr
15m	0.145945	0.225586	0.000418	NaN
30m	0.205102	0.316606	0.000841	NaN
45m	0.249220	0.386037	0.001278	NaN
60m	0.287304	0.443785	0.001715	NaN
75m	0.321510	0.494604	0.002154	NaN
90m	0.352292	0.540447	0.002592	NaN

NAIVE DIRECTION (majority train class per horizon)

	accuracy	macro_F1	DOWN_prec	DOWN_rec	DOWN_f1	NEUTRAL_prec	NEUTRAL_rec	NEUTRAL_f1	UP_prec	UP_rec	UP_f1	n
15m	0.538256	0.233275	0.0	0.0	0.0	0.538256	1.0	0.699826	0.0	0.0	0.0	46686.0
30m	0.522127	0.228683	0.0	0.0	0.0	0.522127	1.0	0.686049	0.0	0.0	0.0	46686.0
45m	0.519021	0.227787	0.0	0.0	0.0	0.519021	1.0	0.683362	0.0	0.0	0.0	46686.0
60m	0.512488	0.225892	0.0	0.0	0.0	0.512488	1.0	0.677675	0.0	0.0	0.0	46686.0
75m	0.514844	0.226577	0.0	0.0	0.0	0.514844	1.0	0.679732	0.0	0.0	0.0	46686.0
90m	0.514351	0.226434	0.0	0.0	0.0	0.514351	1.0	0.679302	0.0	0.0	0.0	46686.0

notebooks > 05_local_training.ipynb > 05 - Local training (laptop / workstation) > 1. Environment > import os

Generate | Code | Markdown | Stop Execution | Go To

Model: "gru_multitask"

Layer (type)	Output Shape	Param #	Connected to
window (InputLayer)	(None, 64, 49)	0	-
gru_1 (GRU)	(None, 64, 64)	22,080	window[0][0]
drop_1 (Dropout)	(None, 64, 64)	0	gru_1[0][0]
gru_2 (GRU)	(None, 64)	24,960	drop_1[0][0]
drop_2 (Dropout)	(None, 64)	0	gru_2[0][0]
direction_logits (Dense)	(None, 18)	1,170	drop_2[0][0]
direction_head (Reshape)	(None, 6, 3)	0	direction_logits[0][0]
return_head (Dense)	(None, 6)	390	drop_2[0][0]

Total params: 48,600 (189.84 KB)

Trainable params: 48,600 (189.84 KB)

Non-trainable params: 0 (0.00 B)

```
shape check: {'return_head': (3, 6), 'direction_head': (3, 6, 3)}
head shapes OK: {'return_head': (3, 6), 'direction_head': (3, 6, 3)}
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 2026-10-16

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	D. S. Weerasooriya	**Registration No.**	**D/DBA/25/0003**
Degree / Intake	BSc (Hons) Data Science and Business Analytics / Intake 42	**Group No.**	**08**
Project Title	Bitcoin Price Forecasting Using Deep Learning Time-Series Analysis (CryptoForecast)		

Technical Contributions:

Task / Component	**My Contribution**	**Tools / AI Technique Used**	**Evidence / Output**
Web dashboard: actual price vs predicted curve chart	Developed the main dashboard chart. It draws candlesticks (actual price) and, on the same time axis, the model's projected close, the no-change baseline, the range of realised outcomes and the model's recorded earlier projections. Added a volume pane on the same time axis and	React 18, Vite 5, lightweight-charts 4.2, JavaScript (JSX); JSON data from the FastAPI server	GitHub commit 21bf140 (web/ folder, 23 files) web/src/components/PriceChart.jsx, VolumeChart.jsx; web/src/markers.js Screenshots: "Actual price vs predicted curve" and "Volume" panels

	markers for right and wrong calls.		
Probability bars and live scorecard charts	Developed the panels that show the model output for each horizon: stacked bars for direction (DOWN / NEUTRAL / UP) and activity (QUIET / ACTIVE) probabilities, and two live scorecard line charts showing the running hit rate of the model's own calls. The two heads are shown separately because their baselines differ.	React (JSX components); direction and regime head outputs of the GRU model	web/src/components/ProbabilityBars.jsx, TrackRecord.jsx Screenshots: "Direction probabilities", "Activity probabilities", "Live activity scorecard", "Live direction scorecard"

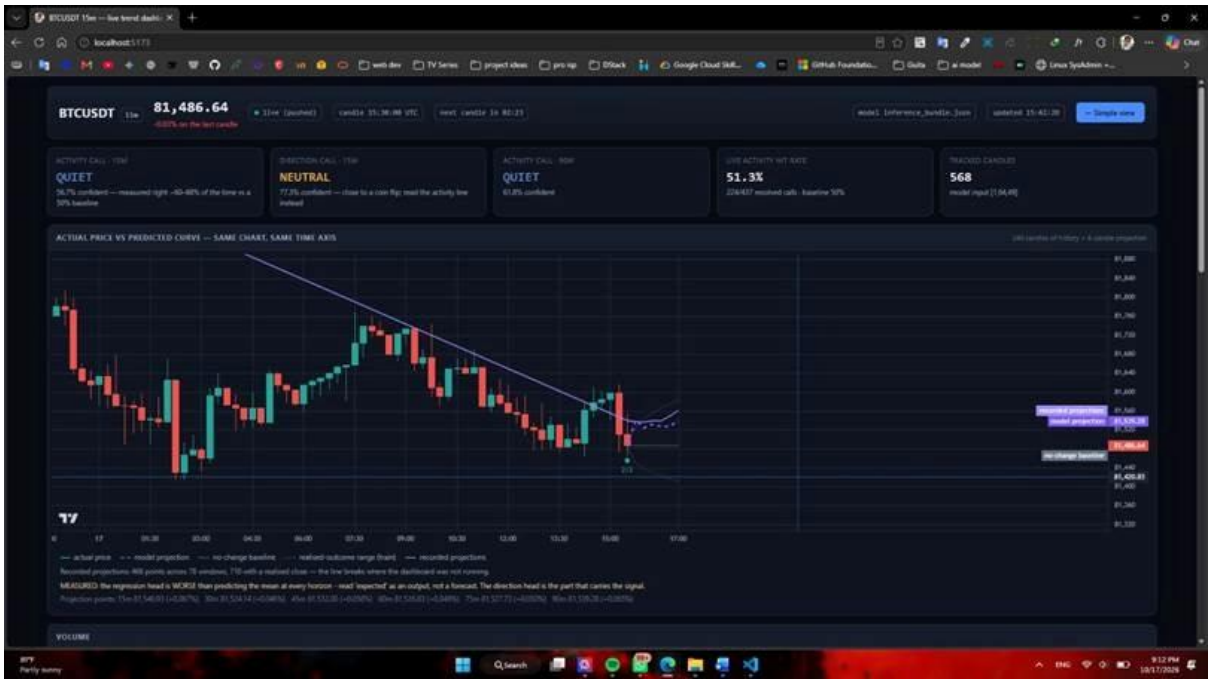
Return distribution chart, results table and call log	Developed the "Realised returns vs what the model expects" histogram (training returns for each horizon, shaded NEUTRAL band, line for the current prediction), the held-out performance table that shows each result next to its baseline, and the call log that lists hits and misses.	React (JSX); recharts bar chart; histogram of training-split returns; baseline comparison	web/src/components/ReturnDist.jsx, MetricsTable.jsx, CallsTable.jsx Screenshots: "Realised returns vs what the model expects", "Held-out performance", "Call log"
Simple view and dashboard checks	Developed the Simple view, which gives two plain-language answers, a chart and a table, and states that the direction answer is close to a coin flip. Wrote the checks that render every panel against a saved API snapshot and test the chart markers. Fixed a chart crash caused by markers that were not in time order.	React; Node.js; esbuild; saved API snapshot (_snapshot.json)	web/src/components/SimpleView.jsx, web/src/App.jsx web/check.mjs, ssr-check.jsx, markers-check.mjs Report Table 6.2 (T11–T13) and Section 5.10

Analysis and interpretation of results	Read the dashboard results against their baselines. Activity accuracy was 64.77% against 49.77%. Direction accuracy was 52.82% against 52.02% (a lift of only 0.8 points), with macro-F1 42.37%. The live activity hit rate was 51.3% (224 of 437 calls), a small sample. The return head was worse than predicting the mean. So the model finds some signal about market activity but very little about direction.	Accuracy and macro-F1 against baselines (majority class and 0.50); calibration; live scorecard	Screenshot: "Held-out performance" table and stat cards Report Chapter 7 (Sections 7.1–7.6)
Terminal dashboard and actual-vs-predicted trend chart	Assisted: captured screenshots of the terminal dashboard (six-horizon read, live scorecard, recent calls) and used the actual-vs-predicted direction chart to explain the NEUTRAL bias of the direction head. The code of these two tools was written by other members.	Python (rich terminal dashboard, plot_trend.py with matplotlib); GRU ensemble	Screenshot: terminal dashboard "live trend read" src/dashboard.py, src/plot_trend.py (local copy) Report Figures 5.5 and 7.8

Progress reporting, report writing and diagrams	Wrote and edited parts of the progress report and the final project report, and prepared figures and diagrams that explain the system: the end-to-end workflow, the chronological data split and the model structure.	Microsoft Word; technical writing; diagram and figure tools	Progress report; final report (Group 8) Report Figures 4.1–4.3, 5.1–5.5 and 7.1–7.8
Existing project work (reviewed only)	Reviewed and understood, but did not develop or train: the data download and cleaning, the 49 features, the GRU model and training, the evaluation scripts and the leakage audit.	Python, TensorFlow / Keras, pandas, scikit-learn (used by other members)	README.md, PROJECT_USAGE.txt, notebooks 01–03

EVIDENCE

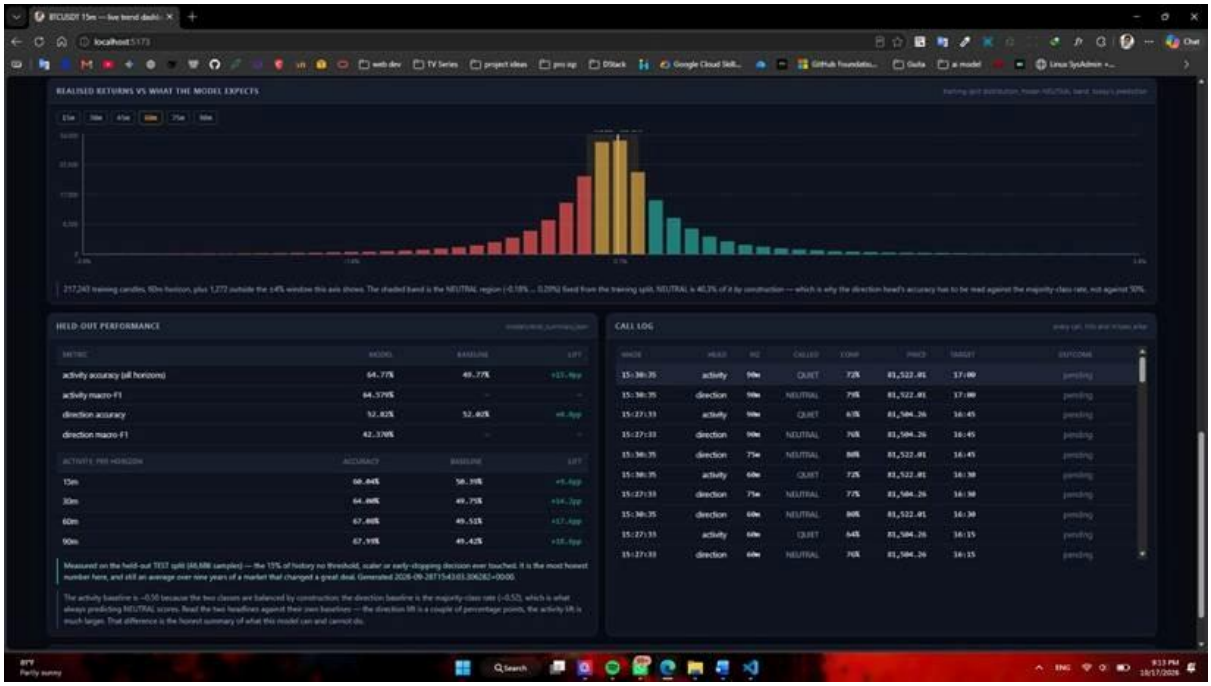
A. Dashboard screenshots (web/ React dashboard)



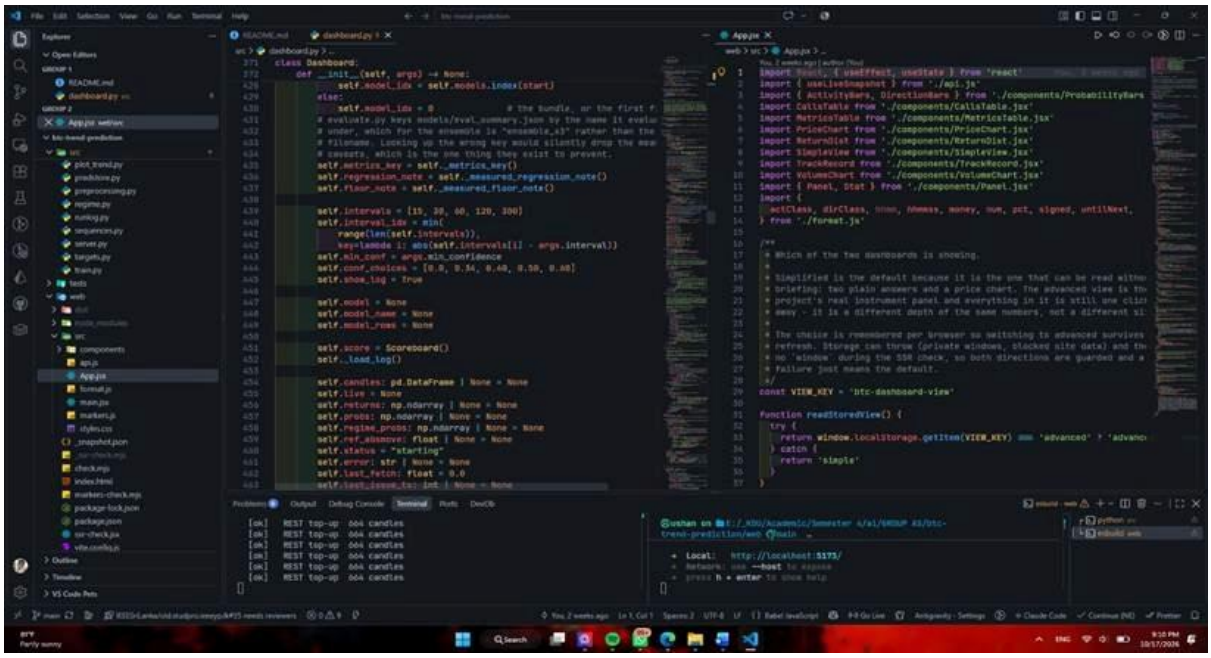
Advanced view – stat cards and "Actual price vs predicted curve" chart



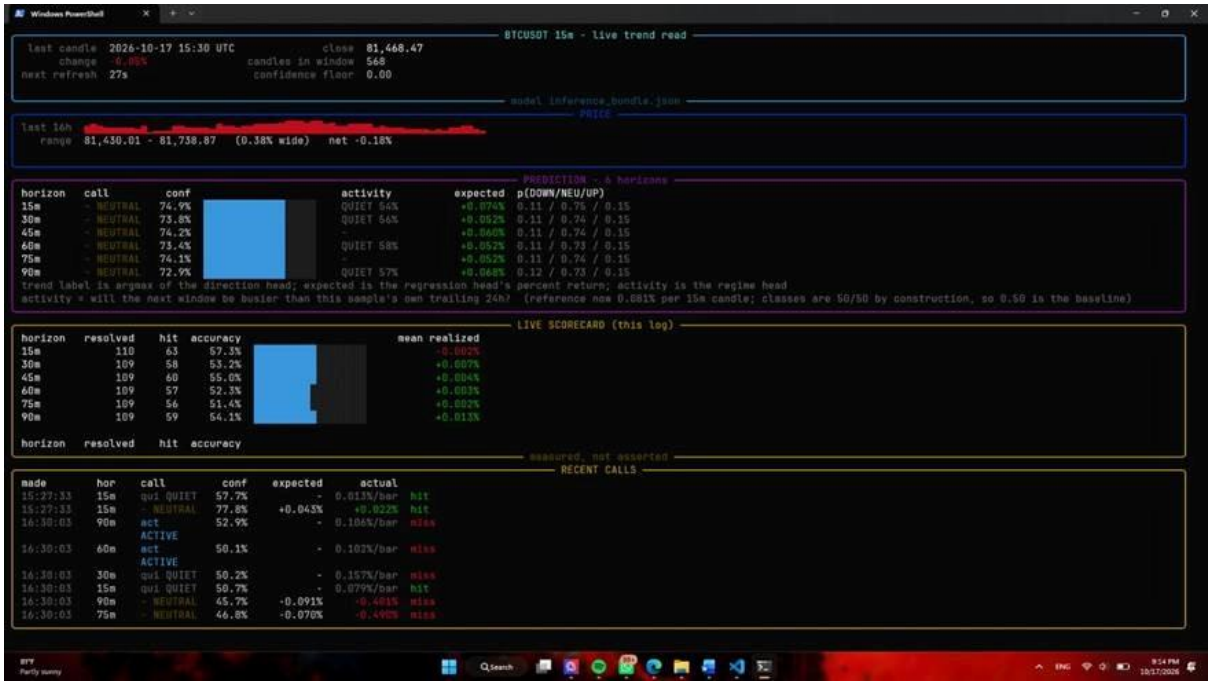
Volume pane, direction and activity probability bars, and the two live scorecards



"Realised returns vs what the model expects" histogram with the NEUTRAL band



VS Code showing the web/src folder



web/src/components/PriceChart.jsx

```
45 +  */
46 + export default function PriceChart({ snapshot, height = 460, simple = false }) {
47 +   const boxRef = useRef(null)
48 +   const chartRef = useRef(null)
49 +   const seriesRef = useRef({})
50 +
51 +   // -- create once ----------------------------------------------------
52 +   useEffect(() => {
53 +     if (!boxRef.current) return undefined
54 +
55 +     const chart = createChart(boxRef.current, {
56 +       autoSize: true,
57 +       layout: {
58 +         background: { type: 'solid', color: C.panel },
59 +         textColor: '#9aa4b8',
60 +         fontSize: 11,
61 +       },
62 +       grid: {
63 +         vertLines: { color: C.grid },
64 +         horzLines: { color: C.grid },
65 +       },
66 +       rightPriceScale: {
67 +         borderColor: C.grid,
68 +         scaleMargins: { top: 0.12, bottom: 0.12 },
```

```
102 +     // behind "now" and the projection is entirely ahead of it.
103 +     seriesRef.current.history = chart.addLineSeries({
104 +       color: C.history, lineWidth: 2,
105 +       lineStyle: LineStyle.Solid, priceLineVisible: false,
106 +       lastValueVisible: false, crosshairMarkerVisible: false,
107 +       title: 'recorded projections',
108 +     })
109 +
110 +     // Advanced-only layers. In the simple view these are never created - not
111 +     // created and left empty, because an empty series still takes a colour in
112 +     // the legend and a reader would be told the chart is drawing more lines
113 +     // than it is.
114 +     if (!simple) {
115 +       // The rails go in before the model line, because lightweight-charts has
116 +       // no z-order API in v4: a series draws over the ones added before it, so
117 +       // insertion order here is the only thing keeping the projection legible
118 +       // against its own error bars.
119 +       seriesRef.current.bandHi = chart.addLineSeries({
120 +         color: 'rgba(124,92,255,0.55)', lineWidth: 1,
121 +         lineStyle: LineStyle.Dotted, priceLineVisible: false,
122 +         lastValueVisible: false, crosshairMarkerVisible: false,
123 +       })
```

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Date: 2026-10-16

Student Signature: 

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.